

Hype Cycle for Agile and DevOps, 2025

31 July 2025 - ID G00826568 - 110 min read

By Nabeeha Ahmed, Manjunath Bhat, [and 1 more](#)

Agile and DevOps are rapidly evolving, but they remain essential pillars of modern software engineering. Software engineering leaders should use this research to identify the innovations that will elevate their teams' performance and drive their organizations forward.

Analysis

What You Need to Know

The agile and DevOps landscape is at a turning point. Despite widespread adoption, many organizations have experienced disappointing results due to superficial implementations and antipatterns, leading to skepticism about their value. The industry is increasingly moving toward product-centric delivery and the product operating model. Now, more than ever, agile and DevOps are expected to be tools to achieve results, rather than be ends in themselves.

It is important to recognize that the core objective of agile and DevOps has always been to enable better business outcomes. The current emphasis on product alignment reflects a natural progression of established practices, highlighting the ongoing refinement of approaches that have supported organizational success for many years.

The real challenge today is not whether to adopt these practices, but how to use them together effectively. True innovation and high-performing teams emerge not from following prescriptive frameworks, but from adapting, iterating and delivering real value in the face of

constant change. Software engineering leaders must move beyond rigid recipes and certifications to focus on experimenting and embracing the principles of agile and DevOps.

The Hype Cycle

A big question for software engineering leaders today is how to supercharge team productivity while staying flexible and adaptable to meet evolving business demands. The answer lies in the interplay of three critical areas: AI, platform engineering and developer experience (DevEx).

AI is increasingly becoming a significant factor in both productivity gains and the emergence of new risks. Platform engineering is emerging as essential for managing the complexity associated with AI. It enables infrastructure automation, workflow standardization, system governance and improved DevEx. At the same time, focusing on DevEx ensures that new platforms and tools are adopted and that top talent is retained. A balanced focus on all three innovations leads to a more effective and successful transformation.

This year's Hype Cycle spotlights these three critical areas, as well as emerging trends, such as micro product teams, the product operating model and vibe coding. Vibe coding, in particular, transforms the developer's role from writing code to expressing high-level goals in natural language, with AI translating these intentions into functional code.

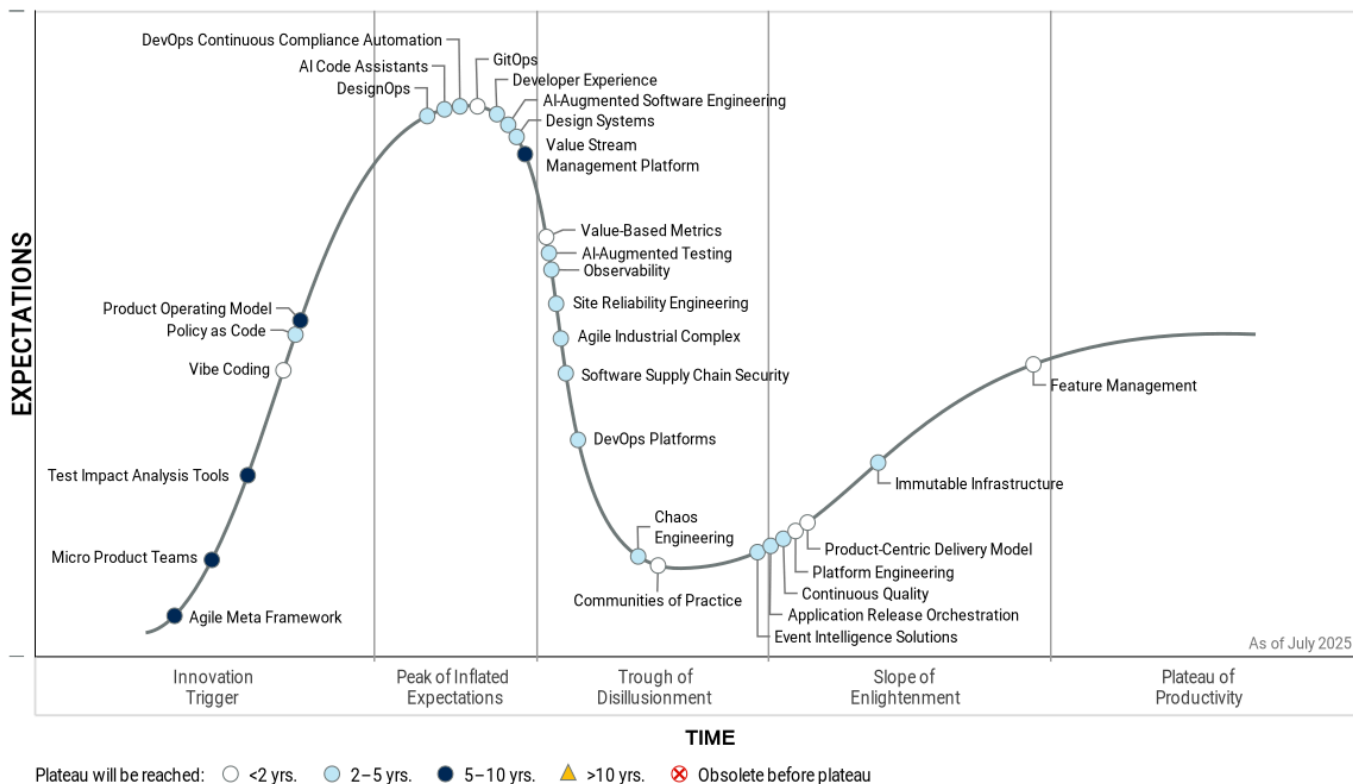
Meanwhile, practices such as continuous quality and application release orchestration are gaining momentum as organizations begin to recognize their potential, understand their practical applications and start to invest in them more seriously.

In 2025, agile and DevOps are no longer just process improvements — they are engines of innovation, powered by AI and intelligent platforms.

Figure 1: Hype Cycle for Agile and DevOps, 2025



Hype Cycle for Agile and DevOps, 2025



Gartner

The Priority Matrix

The Priority Matrix maps the time to maturity of technologies and frameworks in an easy-to-read grid format. It answers two high-priority questions:

- How much value will an organization receive from an innovation?
- When will the innovation be mature enough to provide this value?

On this Hype Cycle, we see platform engineering and vibe coding poised to offer transformational benefits to mainstream organizations within the next two years. High, but not transformational, benefits will arrive soon in the form of communities of practice, feature management, GitOps, the product-centric delivery model and value-based metrics.

Priority Matrix for Agile and DevOps, 2025

Benefit Rating	Business Value	Software Engineering	Product
		AI Code Assistants Observability Site Reliability Engineering Software Supply Chain Security	Feature Model Value Stream Management Platform
High	Communities of Practice Feature Management GitOps Product-Centric Delivery Model Value-Based	AI-Augmented Testing Application Release Orchestration Continuous Quality Design Systems Developer Experience DevOps	

Source: Gartner (July 2025)

On the Rise

Agile Meta Framework

Analysis By: Peter Hyde, Nabeeha Ahmed

Benefit Rating: Moderate

Market Penetration: Less than 1% of target audience

Maturity: Embryonic

Definition:

An agile meta framework offers a common foundation for understanding, describing and improving ways of working in software engineering. It features a shared language,

standardized practice descriptions, method independence and the ability to tailor processes to meet unique team needs.

Why This Is Important

The 2001 Agile Manifesto laid the groundwork for countless models, frameworks and practices, revolutionizing team performance and success. Yet, applying agile methods can be tough, as rigid frameworks often fall short. Achieving business agility demands innovative customization using agile meta frameworks to ensure a fit-for-purpose approach, enabling effective transformation and product success.

Business Impact

By allowing method independence and process customization, agile meta frameworks enable teams to tailor their approaches to specific needs, improving efficiency and adaptability. This flexibility drives successful transformations, enhances product delivery, and ultimately boosts overall business agility and competitiveness. Agile meta frameworks are crucial for creating adaptable, efficient and collaborative environments in software engineering, driving better outcomes and fostering innovation.

Drivers

- **Flexibility demand:** Organizations require adaptive frameworks that can be dynamically tailored to address specific team requirements, organization sizes, evolving product demands and volatile market conditions.
- **Complexity management:** A cohesive agile framework helps streamline processes and reduce dependencies as products become more complex.
- **Business alignment:** Improving the alignment of software development to business objectives ensures a more effective and efficient delivery of customer impact and business value.
- **Platform engineering:** Enterprise platforms with self-service capabilities function more efficiently when consistent, standardized processes are in place. The processes must be flexible and tailorable to meet the needs of individual products.

Obstacles

- **Cultural barriers:** Individuals and teams may resist altering established processes and practices, hindering adoption. Organizational culture, change fatigue and fear of failure can all be factors.
- **Lack of understanding:** Insufficient knowledge of agile meta frameworks can lead to misapplication and ineffective implementation. Transformation success requires clear executive sponsorship and sufficient time for experimentation and learning.
- **Inconsistent adoption:** Integrating new ways of working with legacy workflows can be complex and challenging, leading to uneven adoption and friction across teams.

User Recommendations

- **Assess current ways of working:** Begin by understanding the ways of working across your organization, identifying the challenges that teams experience and the friction points when managing dependencies. Share this analysis to anchor a transparent view of the challenges faced.
- **Explore agile meta frameworks:** Ask your agile coaching group, or software leads when not present, to investigate agile meta framework options, such as the Agile Fluency Model, Essence and Hexi, and propose an optimal path forward.
- **Drive innovation in workflows:** To avoid legacy process integration challenges, invest in training and mentoring for a full product group or business portfolio. Manage the implementation as a change initiative with executive sponsorship to ensure a successful transformation.
- **Encourage experimentation:** Cultivate a robust culture of experimentation and continuous improvement, empowering teams to tailor their practices to fit specific needs and enhance processes.

Gartner Recommended Reading

[The Why, What and How of Successful Agile Transformation](#)

[Solution Path for Agile Transformation](#)

Micro Product Teams

Analysis By: Peter Hyde, Nabeeha Ahmed

Benefit Rating: Transformational

Market Penetration: Less than 1% of target audience

Maturity: Embryonic

Definition:

A micro product team is an AI-native software engineering team comprising a product manager, product designer and software engineer. These talent-dense teams are empowered, autonomous and fully accountable for the success of their product.

Why This Is Important

Generative AI is transforming software engineering, shifting focus from cost-centric development to customer-centric innovation. AI-augmented micro product teams, with their compact size, enable rapid decision making and adaptation. Their talent density ensures high-quality outcomes, while autonomy empowers them to efficiently deliver creative, customer-focused solutions aligned with market demands and user needs with a fast cadence.

Business Impact

Micro product teams have the potential for a transformational impact on organizations. Their cost-efficient structure, faster time to market, increased innovation and enhanced customer focus offer a competitive advantage. Additionally, their flexibility and scalability enhance organizational resilience and responsiveness to market changes, solidifying long-term success.

Drivers

- **Pervasive AI:** Successfully integrating AI throughout product development enhances capabilities, enabling micro product teams to innovate faster and more efficiently, driving interest in their potential.
- **Economic efficiency:** Amid economic challenges, organizations seeking cost-effective solutions find the lean design of micro product teams appealing.
- **Business agility:** The demand for rapid adaptation to evolving market needs and customer expectations emphasizes the value of small, product-focused teams capable of pivoting quickly.

- **Customer centricity:** Micro product teams excel in delivering personalized, customer-centric solutions, ensuring products are closely aligned with rapidly changing customer needs and preferences.

Obstacles

- **Organizational resistance:** Resistance from those accustomed to established ways of working is common when shifting from a traditional hierarchical structure to autonomous product teams.
- **Skill gaps:** Fulfilling the need for highly skilled, multidisciplinary team members is challenging, potentially limiting the effectiveness of micro product teams.
- **Scaling risks:** Managing enterprise-scale operations with many micro product teams, particularly when dealing with interconnected dependencies, poses a significant challenge to widespread adoption.
- **Balanced governance:** Empowering teams while maintaining strategic oversight and strong governance can be difficult, affecting decision making and accountability.

User Recommendations

- Transition to a product operating model with dedicated product teams that deliver rapid customer value securely and independently, incorporating rapid end-user feedback.
- Embed AI capabilities into all roles on product teams to enhance automation, boost efficiency, drive innovation and generate practical value.
- Prioritize skill development to equip team members with the expertise needed to innovate and meet the diverse demands of talent-dense teams.
- Select a candidate for a micro product team proof of concept to assess the approach's effectiveness within your organization. Set clear objectives and provide robust support throughout the team's creation and evolution.
- Conduct a comprehensive analysis of the team's achievements by evaluating their impact on business results to demonstrate the effectiveness of their work.

Gartner Recommended Reading

[Leadership Actions for a Successful Project-to-Product Transition](#)

Test Impact Analysis Tools

Analysis By: Jim Scheibmeir, Joachim Herschmann, Thomas Murphy

Benefit Rating: Low

Market Penetration: 1% to 5% of target audience

Maturity: Emerging

Definition:

Test impact analysis tools determine a subset of affected tests from all the available tests on the basis of application areas that are impacted by the code changes. Test impact analysis optimizes test execution time by running only a subset of tests without compromising the quality of the application under test.

Why This Is Important

Testing is always a risk-based activity and the fact that testers cannot test everything, even when using automation, will usually result in trade-offs and compromises. Test impact analysis tools help to minimize risk by optimizing test sets, increasing test coverage, selecting and prioritizing critical tests based on contextual information and reducing the cognitive load on the testers.

Business Impact

Test impact analysis tools mean less testing without sacrificing coverage of the code change. Reducing the initial test suite size accelerates feedback. Reducing a test plan's size involves selecting the relevant regression test scripts for a release based on information about code changes, features and bugs fixed. Removing duplicate test cases by identifying redundancies and similarities in test-case inventories improves test execution sequencing.

Drivers

- The growing complexity of systems and the need for efficient testing are driving the adoption of test impact analysis tools.

- As applications become feature-rich, they also become more complex, making effective testing crucial for digital business success.
- Developers require fast feedback so they can stay within their flow of work, complete their definition of done and reduce context-switching waste due to slow testing finding latent bugs late in the life cycle.
- By reducing the test suite size and removing duplicate test cases, test environments are used for a shorter amount of time, which opens up critical resources when the environments are shared.
- Software engineering leaders and their teams are eager to find technology and practices that help them build confidence regarding their systems and inform accurate release decisions.
- These tools also aid in selecting and prioritizing critical tests based on contextual information, improving decision making for software release.

Obstacles

- **Limited budget:** Organizations may not have the resources to invest in these tools, especially if they are already committed to other tools or services.
- **Organizational culture:** If an organization is resistant to change, it may be difficult to convince stakeholders of the need to “test less.”
- **Lack of awareness:** Some organizations may not be aware of the benefits of these tools or may not understand how they work.
- **Slow decision making:** Bureaucracy can delay the adoption of new tools.
- **Lack of training:** If staff does not have the necessary training to use these tools effectively, they may be reluctant to adopt them.
- **Integration challenges:** Test impact analysis tools may not integrate well with existing systems or workflows, which can create obstacles for adoption.
- **Data privacy concerns:** Some organizations may have concerns about data privacy and security when using these tools, especially if they involve cloud-based or third-party services.

User Recommendations

- Test impact analysis tools work best with software engineering teams that already have a continuous integration/continuous delivery (CI/CD) pipeline in place. The accuracy of the test impact models is dependent upon the type and amount of data they have access to, such as code repositories and application performance monitoring tools.
- Engage your security team to evaluate the risks of integrating these tools. Set proper expectations that the time it takes to gather accurate test impact analysis will depend on the amount of data access that the tools can utilize to properly build accurate risk models for your applications.
- Evaluate how long CI/CD pipelines are taking to get proper feedback to developers regarding software quality. When that task is taking longer than 20 minutes, pilot teams with this technology.

Sample Vendors

Appsurify; CloudBees; Drill4J; LambdaTest; Microsoft; OpsHub; Tricentis

Gartner Recommended Reading

[Market Guide for AI-Augmented Software-Testing Tools](#)

[Shift Quality Further Left With Behavior-Driven Development](#)

[Innovation Insight for Test Impact Analysis](#)

[Essential Skills for Quality Engineers](#)

Vibe Coding

Analysis By: Bill Blosen, Peter Hyde

Benefit Rating: Transformational

Market Penetration: 1% to 5% of target audience

Maturity: Emerging

Definition:

“Vibe coding” is a term coined in February 2025 by computer scientist Andrej Karpathy. It transcends AI-augmented development tools to envision a new state of human-computer interaction. Developers become composers, using voice recognition or light keyboarding to rapidly prototype complex yet throwaway, not-for-production software. Vibe coding ignores the generated code, focuses on results and has AI solve all bugs. Thus, the developer stays in a high productivity state known as “flow.”

Why This Is Important

Generative AI (GenAI)-augmented coding is a paradigm shift in how software is created and managed. Vibe coding builds on that and is a harbinger of the future of software programming. Much hype is being generated, and it is important to understand it clearly, as there are profound implications; the advancement of low-code, AI coding assistants and vibe coding may alter the need for traditional professional coding. In addition, the risks of implementing this improperly are very high.

Business Impact

Vibe coding must be considered by enterprises that prioritize innovation and rapid learning cycles with focus group customers. As tools develop and are trained on reliable and secure codebases, vibe coding will become viable and is poised to give new and experienced engineers a massive gain in innovation, creativity and productivity.

Drivers

- **GenAI technology advances:** State-of-the-art, frontier GenAI models from Anthropic and OpenAI have enabled developers to implement solutions in a fraction of the time they previously spent looking things up on Stack Overflow or Google. Many AI coding assistants now support vibe coding through implementing new agentic capabilities that allow the developer to easily compose complex software through a flow-based conversation.
- **Desire for rapid customer feedback:** Enterprises are increasingly focused on accelerating the cycle of delivering software to generate learning from customers. Vibe coding and fast cycles like lean startup are a natural fit for early-stage venture-capital-funded startups.

- **Repository indexing:** As AI tools become more refined, the quality of source code used to prompt or tune them will improve. Fine-tuning with your own codebases allows the tools to integrate your unique architecture, security and reliability patterns into the software produced by vibe coding.
- **The search for flow:** Software developers value having time to focus on their core work, but often do not have enough of it. Flow state is a high-productivity mental state in which a person is completely focused on their work. Vibe coding keeps a developer in this state by continually focusing on creating functionality without any need to fix bugs, create tests or review the code.
- **Voice technology advances:** Software coding involves significant amounts of typing and mousing to write and manage code. Voice recognition and text-to-speech technologies have now advanced to provide an even further detachment from coding distractions and rote tasks, thus allowing easier access to flow states.

Obstacles

- **Code quality:** The code produced by vibe coding is in no way intended for production use today. Tools supporting vibe coding use GenAI models trained on internet code and focus only on creating functionality without attention to quality principles, like testing and code review. Over time, this will fade.
- **Risks:** Unlike production code, little is known about the contents of the vibe code. Until a track record is established, Gartner recommends using this code only for prototyping and creative innovation.
- **Resistance:** With the rapid pace of change, many workers today are experiencing change fatigue. Although developers enjoy flow state and nice tools, their resistance to more change may overpower it. Developers also take pride in their work and may resist the risks of vibed code.
- **Limited gain:** The resulting code is useful only for innovation experiments. Also as coding is a small portion of the full software development life cycle, the risk may outweigh the benefits.

User Recommendations

- Do not use vibe-coded software in production. Limit it to a controlled, safe sandbox for execution. Use platform engineering principles to provide a safe paved road for engineers to use vibe coding tools and techniques.
- Prioritize AI code assistant tools that incorporate agentic flows with excellent context awareness through RAG or fine-tuning capabilities.
- Enable pilot teams to explore by supporting an innovation culture. Find engineers searching for flow. Lean startup and dedicated innovation teams will easily adopt these techniques. Consider product areas that have rapid feedback opportunities, like customer-facing interactions where you can find willing beta audiences.
- Build a strong integration strategy, libraries of modular components and an API catalog to ease composing programs with vibe coding.

Sample Vendors

Anthropic; Anysphere; Lovable; Microsoft; OpenAI; Replit; StackBlitz; Windsurf

Gartner Recommended Reading

Magic Quadrant for AI Code Assistants

Innovation Insight: Open Generative AI Models for Coding

How to Improve Developer Effectiveness Using AI Coding Tools

Policy as Code

Analysis By: Paul Delory

Benefit Rating: High

Market Penetration: 5% to 20% of target audience

Maturity: Emerging

Definition:

Policy as code (PaC) languages express governance and compliance rules as code, so they can be enforced programmatically by automation tools. PaC languages are often domain-specific and declarative. With PaC, policies are treated as software, making them subject to

version control, code review and functional testing. The most mature PaC tools can render any business logic in code. You can use PaC today to enforce infrastructure compliance, authorization, Kubernetes admission control and more.

Why This Is Important

Platform engineers use PaC to build optimization, governance and compliance controls into automation pipelines. Infrastructure and security teams have used it for years to build guardrails around infrastructure and data while preserving a separation of duties that mirrors a typical IT organization chart. With the rise of generative AI (GenAI), PaC is poised to become a way to control AI agent behavior and enforce standards programmatically, which current GenAI tools often struggle to do.

Business Impact

- **Security, compliance and automation:** PaC, combined with automation, enforces policies with implicit compliance guarantees.
- **Alignment of security and operations teams:** PaC allows security and compliance teams to interface directly with automation pipelines.
- **Visibility and auditability:** PaC documents policies. PaC tool logs can be audited to prove policies are being enforced.
- **Time and effort spent:** PaC means less toil for operators because it forestalls configuration drift and out-of-spec elements.

Drivers

- **Emerging standard:** Several dedicated PaC tools are now on the market, many of them open source. The Open Policy Agent (OPA), a Cloud Native Computing Foundation project, has become the de facto standard for PaC. Even some other PaC tools now use OPA policies alongside or instead of their own policy engines.
- **Increasing regulations:** Regulations such as General Data Protection Regulation have increased both the difficulty of compliance and the pressure on compliance teams. PaC allows compliance teams and auditors to document their policies in detail and verify that they are being enforced.

- **Agentic AI:** The advent of GenAI agents is transformational across almost every industry, but organizations struggle to control the behavior of AI agents. PaC can provide both effective control and meaningful testing and auditing of agents' outputs.
- **Security breaches:** A spate of newsworthy security breaches at public companies — caused by infrastructure misconfigurations — has put every IT organization's security and compliance practices under increased scrutiny. No infrastructure and operations team wants its security failures to be the reason its company gets negative headlines.
- **Continued growth of DevOps and DevSecOps:** As more companies are embracing DevOps and DevSecOps, they are also encountering the hard governance problems of automation. Many teams that implement infrastructure as code quickly find that they need better policy enforcement, and PaC can help.
- **Cloud optimization and cost control:** Besides their benefits for security and compliance, PaC tools can also be used to enforce the build standards for infrastructure, including budgets. In the public cloud, where oversized or unnecessary infrastructure incurs direct out-of-pocket costs, programmatically enforced policies can help to control spending.

Obstacles

- **Scarcity of downloadable content:** PaC tools will not gain real traction until they have extensive libraries of community-generated content from which users can download the policies they need rather than having to write their own. Over time, as the user base expands, PaC tools will reach a critical mass of downloadable content that supports real-world uses.
- **Skill set:** Many technical professionals lack the skills to operate automation and PaC tools effectively. As the learning curve might be steep for some, you may need to accept some flubbed policy enforcement due to lack of experience.
- **Integration challenges:** Integrating with existing tools is complex and often requires additional configuration.
- **Organizational inertia:** In some organizations, collaboration between infrastructure and operations teams and security or compliance teams is actually unwanted. This dynamic may slow the rate, scope and scale of PaC initiatives.
- **Costs:** Even if PaC tools themselves are free, you may still require training or consulting.

User Recommendations

- **Start small:** Choose a pilot use case where PaC will likely provide real business benefits, then expand to others once PaC has proven its value.
- **Upskill staff:** PaC languages are not always intuitive. Technical staff will need practice and/or training. Adopt the four-eyes principle to prevent flawed policies from impacting operations.
- **Promote reusability:** Focus your PaC efforts on use cases that have ready-made implementation templates — ideally, downloadable content. For example, almost every PaC tool on the market has a canned implementation of the customer information systems benchmarks.
- **Break down team silos:** Use PaC to build a common workflow for automation and policy enforcement that spans platform engineering, infrastructure and operations, security, and compliance teams.
- **Integrate PaC into automation pipelines:** Use PaC to build guardrails for automation tools, so that they cannot take actions that are out of compliance.
- **Measure before and after:** Use observability tools and value stream mapping to define your starting state, then compare it to the end state. Collect real data to quantify the value of PaC.

Sample Vendors

IBM (HashiCorp); Palo Alto Networks; Progress; Pulumi; Styra

Gartner Recommended Reading

[Using 'Policy as Code' to Secure Application Deployments and Enforce Compliance](#)

[How to Protect Your Clouds With CSPM, CWPP, CNAPP and CASB](#)

[Innovation Insight for Continuous Compliance Automation](#)

[Innovation Insight for Cloud-Native Application Protection Platforms](#)

[Magic Quadrant for DevOps Platforms](#)

Product Operating Model

Analysis By: Peter Hyde, Nabeeha Ahmed

Benefit Rating: High

Market Penetration: 20% to 50% of target audience

Maturity: Adolescent

Definition:

A product operating model defines how organizations structure and manage product development, aligning resources and strategies to deliver customer value. It encompasses governance, processes and technology, fostering agility, collaboration and continuous improvement for effective product delivery.

Why This Is Important

The ongoing transition from projects to products in software engineering underscores the importance of a strong customer-centric focus, long-term value creation and increased business agility. Empowered product teams drive innovation and make more effective decisions. Products that align with market demand achieve higher satisfaction and user loyalty, bolstering market position and fostering business growth and competitive advantage.

Business Impact

The product operating model significantly impacts businesses by driving innovation, enhancing customer focus and ensuring strategic alignment with business objectives. Dedicated product teams and rapid customer feedback boost innovation, enabling swift responses to new opportunities and challenges. A strong customer focus results in superior user experiences and strengthens market positions. Strategic alignment ensures all product efforts support business objectives and deliver on key priorities.

Drivers

- **Demand for agility:** Organizations seek techniques that enable rapid adaptation to market changes and technological advancements, making the product operating model appealing.

- **Customer centricity:** Increasing emphasis on delivering personalized and user-focused solutions highlights the model's effectiveness in aligning products with customer needs.
- **Digital transformation:** The pace required for software engineering modernization necessitates continuous innovation and the efficient integration of new technologies.
- **Competitive pressure:** Businesses face intense competition, driving the adoption of operating models that enhance innovation, efficiency and business alignment.

Obstacles

- **Cultural resistance:** Efforts to shift from a traditional project-based approach to a product-focused model often face pushback from employees accustomed to established workflows and practices.
- **Skill gaps:** Teams may lack the skills and experience to successfully implement and operate within a product-focused framework, necessitating a substantial investment in skill development and time to experiment.
- **Outmoded design:** Existing hierarchical structures, traditional portfolio management, legacy monolithic architectures and limited platform technologies can hamper the formation of autonomous product teams.

User Recommendations

- Start a change initiative for moving to a product operating model with strong support and commitment from leadership to drive the necessary cultural and structural changes.
- Invest in skill development to equip teams with the expertise required to operate within a product-focused framework. Example skills include product management and DevOps practices.
- Redesign your organizational structure to establish autonomous product teams enabled by platform engineering, architectural design and automation practices.
- Build a customer-centric culture focused on delivering high-quality products and optimizing user experience based on data. Continuously iterate and refine products by actively incorporating customer feedback, ensuring ongoing enhancement and alignment with user needs.

Gartner Recommended Reading

[Leadership Actions for a Successful Project-to-Product Transition](#)

[Product Owner Essentials](#)

At the Peak

DesignOps

Analysis By: Will Grant, Brent Stewart

Benefit Rating: Moderate

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

Design operations, or DesignOps, is a discipline that focuses on the process and production aspects of design, rather than the creative aspects. DesignOps practices guide the design process, coordinate teams, manage resources, and ensure that the design aligns with the business objectives, product management and technology operations. The goal is to enable efficient and DevOps-compatible plans, estimates and processes that support user experience (UX), collaboration and ongoing innovation.

Why This Is Important

DesignOps introduces formalized approaches to governance, operations and people management within design. As a set of easy-to-use operational standards, DesignOps continues to gain in popularity. Digital product companies and agencies are discovering the tremendous value of a proven operational approach for UX team management and design delivery on product teams.

Business Impact

DesignOps represents the first widespread implementation of operational methods and techniques created for both designers and developers. DesignOps adds value during the creation and delivery of design assets. DesignOps practices can support ongoing feature

enhancement and idea generation without interrupting the continuous workflow of development teams.

Drivers

- **Innovation:** When coupled with DevOps, DesignOps leads to more innovative solutions. As a practice, DesignOps employs dual-track agile, which sets aside ongoing tracks of work dedicated to new discovery, idea generation and design exploration. This work acts as a constant source of evidence-based, multidisciplinary innovation.
- **Speed:** DesignOps reduces the time to market for major updates and incremental feature enhancements alike. Due to the concepts of continuous discovery and continuous delivery, developers engage in tech design, architectural explorations and proofs of concept earlier in the process and with a deeper understanding of the overall vision.
- **Collaboration:** DesignOps increases communication and collaboration between designers and developers. DesignOps promotes multidisciplinary teams in workshop settings, design sprints or one-on-one “pairing and sharing” that promotes understanding, empathy and relationship building between these two crucially important groups.
- **Visibility:** Without DesignOps, teams often plan their work in silos. DesignOps encourages the use of a single source of truth for planning, estimating and allocating work — ensuring that all parts of the product team can see the project status at a glance.
- **Remote first:** The COVID-19 pandemic accelerated the shift to remote work, and DesignOps has adapted to this change by increasing the use of digital collaboration and design tools for regular communication, feedback and design consistency.
- **Increased role specialization:** Within DesignOps, roles are becoming more specialized, such as design program managers and design systems managers. These roles focus on specific aspects of operations, allowing for more targeted improvements.
- **Design-to-development workflow:** DesignOps facilitates a seamless transition from design to development by establishing clear workflows and processes. This ensures that design specifications are accurately translated into development tasks, reducing miscommunication and rework.

Obstacles

- **Scarce agile skills:** Few UX practitioners are trained in using a common work breakdown structure for detailed planning and estimation.
- **Low support for design planning:** Few product managers are trained in UX planning, estimating and tracking, and many of the design platforms lack robust change control solutions, although this is improving.
- **Legacy tooling:** Popular enterprise agile planning tools are not designed with UX practitioners, activities and deliverables in mind (although this, too, is improving), leading to resistance from UX teams to adopt tools they feel are “for developers.”
- **Resource constraints:** Establishing a DesignOps function requires investment in tools, training and potentially new roles. Organizations with limited resources may struggle to allocate the necessary budget and personnel.

User Recommendations

- Adopt a DesignOps practice to better manage the complete design life cycle.
- Ensure that the DesignOps practice covers the following four key aspects: how UX teams are organized, the tools and processes for delivering UX artifacts, how design work is planned and tracked, and how success is measured.
- Determine the value of a DesignOps approach with a pilot program involving an existing high-performing team.
- Engage in a productwide rollout that involves training, updated product plans, and the allocation of one or more people to the role of design manager.
- Ensure a successful rollout of DesignOps at the product level by gaining buy-in from product management, design and development teams, and securing robust logistical and administrative skills.

Gartner Recommended Reading

[How Design, Development and Product Management Can Work Together Successfully](#)

[The 4 Secrets of Design-Led Companies](#)

[Quick Answer: How to Build a User Experience Team Business Case](#)

Keys to DevOps Success

AI Code Assistants

Analysis By: Arun Batchu, Philip Walsh, Haritha Khandabattu, Matt Brasier, Keith Holloway

Benefit Rating: Transformational

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

AI code assistants are tools that assist in generating, testing and analyzing software code and configuration. They use foundation models like large language models (LLMs), program-understanding technology, or both. They integrate with developer environments, code editors, command-line terminals, chat interfaces, project management tools, testing, monitoring, logging and deployment tools. Some of them can be customized to an organization's specific codebase and documentation.

Why This Is Important

AI code assistants significantly enhance a software developer's experience by offering capabilities such as faster boilerplate code tasks, identifying code mistakes, writing unit tests and providing comprehensive code explanations. They play a crucial role in code modernization, ensuring applications remain up-to-date with the latest technologies and practices. By boosting efficiency and accelerating application development, these tools minimize cognitive overload, amplify problem-solving skills and maintain the developer's state of flow, fostering creativity and enabling faster learning.

Business Impact

AI code assistants enable faster developer onboarding, accelerate upskilling and provide new ways to solve problems. Developers using AI code assistants improve unit test coverage, reduce technical debt and speed up time to market. Enhanced developer experience and retention are also key benefits.

Drivers

- **Advanced AI capabilities:** Multimodal LLMs and code-specific small language models (SLMs) are enhancing AI-code-assistant functionality, leveraging techniques like retrieval-augmented generation (RAG) and expanding context windows for improved code generation and understanding.
- **Urgent need for productivity:** Software engineering leaders are driven to adopt AI code assistants to increase developer productivity and are focusing on broad adoption to achieve tangible gains. The relatively low subscription costs support this investment.
- **Developer-led adoption:** Developers are embracing these tools in personal projects and expect similar benefits at work, creating demand within organizations. AI enhances developer experience through improved efficiency and reduced cognitive load.
- **Growing vendor market:** The increasing number of vendors reflects and caters to the growing demand for AI code assistants.
- **Rapid innovation via competition:** Intense competition among vendors fuels rapid innovation and frequent feature updates, accelerating adoption.
- **Democratized skills:** AI code assistants facilitate faster learning for individuals with nontraditional backgrounds, expanding the potential talent pool.
- **Empowering citizen developers:** The tools' ability to aid code generation suggests potential for citizen developers to create custom applications.

Obstacles

- Risk of poor quality code and automation bias.
- Generative AI (GenAI) models used by AI code assistants reinforce confirmation bias by suggesting fixes without challenging current solutions, leading developers to write code from scratch instead of using existing libraries and increasing maintenance overhead.
- Rapidly evolving GenAI platform technology offers new interaction methods and workflow integrations, but risks quick obsolescence for today's adopted technologies.
- Frontier models are complex to retrain, limiting updates and restricting use to features and frameworks available during initial training.
- Unrealistic productivity expectations and initial underutilization.

- Amplification of immature practices by novice developers.
- Software development life cycle (SDLC) inefficiencies can limit productivity gains.
- Difficulty in proving clear ROI to finance because of not having a baseline for productivity.
- Possible erosion of core coding skills. Overreliance on AI, leading to confirmation bias.
- Generation of incorrect or hallucinated outputs.

User Recommendations

- Provide guidance on productivity gains for various use cases, not just coding speed.
- Encourage AI assistant use beyond code generation, including documentation, testing, debugging, and refactoring across the SDLC.
- Maintain human involvement to mitigate risks in pair programming and code reviews, encouraging challenges to each other and the AI model for better solutions.
- Offer strategies and tools to manage security, IP issues, bias and ethics in generated code with tracking mechanisms.
- Define and track metrics beyond time savings, covering developer experience, code quality and delivery speed, using frameworks like DORA and SPACE.
- Implement proof-of-concept programs with diverse teams, clear objectives and key results, and comprehensive feedback mechanisms.
- Share examples and best practices for prompt engineering tailored to specific tasks and technologies within your organization.
- Adapt SDLC workflows to integrate AI code assistants and maximize their benefits.
- Develop learning plans to ensure developers effectively use the tools and understand their limitations.
- Tailor adoption strategies based on your organization's development practices, expertise and risk appetite.

Sample Vendors

Alibaba Cloud; Amazon Web Services; Anysphere (Cursor); Exafunction (Windsurf); GitHub; GitLab; Google; IBM; Sourcegraph; Tabnine

Gartner Recommended Reading

[Magic Quadrant for AI Code Assistants](#)

[Critical Capabilities for AI Code Assistants](#)

[How to Communicate the Value of AI Code Assistants](#)

[How to Improve Developer Effectiveness Using AI Coding Tools](#)

[Quick Answer: Mitigating the Top Five Security Risks of AI Coding](#)

DevOps Continuous Compliance Automation

Analysis By: Daniel Betts, Chris Saunderson

Benefit Rating: Moderate

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

Organizations adopting agile, DevOps, DevSecOps and platform engineering struggle to quickly demonstrate compliance requirements across their workflows. DevOps continuous compliance automation (DCCA) tools address this by automatically enforcing guardrails, identifying policy gaps, and audit security and compliance across product and platform delivery life cycles.

Why This Is Important

DevOps organizations need to align with an increasingly large number of regulatory requirements that are evolving at differing rates, and will continue to do so as more compliance requirements are introduced. These requirements are expanding beyond the traditional financial, health privacy and personal privacy to include cybersecurity and contractual mandates plus government regulations.

Business Impact

Continuous compliance automation tools in DevOps enable organizations to achieve, sustain and report on compliance as part of their delivery pipelines and platforms. DCCA tools improve consumer audit reports by:

- Enabling automated enforcement and assessment of security and compliance policies as part of application and infrastructure workflows
- Enabling secure, change-managed toolchains
- Generating audit reports efficiently
- Recommending remediation activities
- Publishing to audit consumers. This reduces the risk of compliance violations, which can result in fines, penalties and reputational damage

DCCA tools also identify compliance gaps and security vulnerabilities early in the development process, when they are less expensive and easier to fix. This prevents costly rework and delays later in the cycle.

Drivers

- As organizations face an increasing number of regulatory obligations and more stringent reporting and enforcement, automating compliance will become even more valuable in maximizing flow and managing cognitive load.
- Regulatory compliance requirements and contractual obligations are growing and evolving. There is an expectation that they must be supported with limited delay, while minimizing the impacts to the flow of customer value.
- The pressure to deliver software faster and more frequently has accelerated development cycles. Traditional, manual compliance processes can't keep pace with this speed, making automation essential.
- Compliance activities are increasingly executed through automated testing for developer efficiencies, change management, segregation of duties and access controls.
- Multiple DevOps toolchains as part of a DevOps initiative often all require compliance insights and must be under compliance control.

- Integrating DevOps workflows into GRC platforms is needed to ensure visibility into compliance levels. As cloud-native application architectures and development models become more pervasive, integrating compliance into the toolchain will become more expected.
- Traditional compliance reporting, benchmarking, assessments and remediation are increasingly too slow to support the needs of high-velocity digital business processes.

Obstacles

- Failure to engage with government risk and compliance (GRC) and compliance and security subject matter experts early in the development life cycle can lead to problems such as poor understanding of policies and their effective implementation.
- Integrating DCCA tools into existing DevOps pipelines can be complex, especially if those pipelines are already intricate or use a variety of tools.
- DCCA tools require a formal change-controlled, secure DevOps toolchain to enable effective auditing.
- A lack of rule-set understanding and consistent implementation can be an impediment to DCCA tool adoption. Failure to consistently involve organizational compliance teams in implementation leads to a failure in delivering maximum value.
- Poorly implemented DCCA presents a business risk. If it is assumed that by implementing DCCA, delivered software becomes compliant without additional effort, organizations will face increased risk of compliance failure.
- DCCA tools can sometimes generate false positives (flagging issues that aren't actually problems) or false negatives (failing to detect real issues). Managing these inaccuracies requires careful tuning and validation.

User Recommendations

- Collaborate on design, implementation and ongoing risk-based approach strategy to filter and prioritize with key stakeholders, including internal GRC, audit, compliance and security.
- Optimize the system of work, taking into account the security and compliance requirements, versus trying to bolt them on after the fact which reduces effectiveness,

efficiency, increases costs, slows things down.

- Implement a “shift-left” approach to ensure compliance controls and evidentiary data are understood and applied earlier in the development process.
- Implement automated compliance checks at every phase of the pipeline, demonstrating a “shift-secure” approach.
- Integrate functions into GRC platforms that provide DevOps compliance and security insights to security and compliance stakeholders.
- Provide an augmented continuous approach to prevent, detect and correct audit failures, and remove manual reporting activities.
- Enable efficient compliance policy checking to measure benchmarks, perform assessments and report on compliance policy controls.
- Research and compare different DCCA tools, considering features, integration capabilities, cost and vendor support. Choose tools that integrate seamlessly with your existing CI/CD pipeline, security tools and other relevant systems.
- Provide thorough training to development, security and compliance teams on how to use the DCCA tools and interpret the results.

Sample Vendors

Anecdotes; CloudBees; Drata; Hyperproof; letsbloom; RegScale; Sprinto

Gartner Recommended Reading

[**Market Guide for DevOps Continuous Compliance Automation Tools**](#)

[**3 Essential Steps to Enable Security in DevOps**](#)

[**Magic Quadrant for Application Security Testing**](#)

[**Using ‘Policy as Code’ to Secure Application Deployments and Enforce Compliance**](#)

[**Magic Quadrant for DevOps Platforms**](#)

GitOps

Analysis By: Paul Delory, Arun Chandrasekaran

Benefit Rating: High

Market Penetration: 20% to 50% of target audience

Maturity: Adolescent

Definition:

GitOps is a type of closed-loop control system for cloud-native applications. According to the canonical OpenGitOps standard, the state of any system managed by GitOps must be expressed declaratively, versioned and immutable, pulled automatically, and continuously reconciled. The term “GitOps” is often used more expansively, usually as shorthand for automated operations or continuous integration/continuous deployment (CI/CD), but this is incorrect.

Why This Is Important

GitOps can be transformational. GitOps workflows deploy a verified and traceable configuration (such as a container definition) into a runtime environment, bringing code to production with only a Git pull request. All changes flow through Git, where they are version-controlled, immutable and auditable. Developers interact only with Git, using abstract, declarative logic. GitOps extends a common control plane across Kubernetes (K8s) clusters, which is increasingly important as clusters proliferate.

Business Impact

By operationalizing infrastructure as code, GitOps enhances management and resilience of services:

- GitOps can improve version control, automation, consistency, collaboration and compliance.
- Configuration of clusters or systems can be updated dynamically. All of this translates to business agility and a faster time to market.
- GitOps resource declarations are version-controlled, modular and stored in a central repository, making them easy to reuse, verify and audit.

Drivers

- **Kubernetes adoption and maturity:** GitOps must be underpinned by an ecosystem of technologies, including tools for automation, infrastructure as code, CI/CD, observability and compliance. Kubernetes has emerged as a ready-made foundation for GitOps, because the continuous reconciliation loop at the heart of K8s complements the GitOps model. As Kubernetes adoption grows within the enterprise, GitOps can, too.
- **Need for increased speed and agility:** Speed and agility of software delivery are critical metrics that CIOs care about. As a result, IT organizations are pursuing better collaboration between infrastructure and operations (I&O) and development teams to drive shorter development cycles, faster delivery and increased deployment frequency. GitOps is the latest way to drive this type of cross-team collaboration.
- **Need for increased reliability:** Speed without reliability is useless. The key to increased software quality is effective governance, accountability, collaboration and automation. GitOps can enable this through transparent processes and common workflows across development and I&O teams. Automated change management helps to avoid costly human errors that can result in poor software quality and downtime.
- **Talent retention:** Organizations adopting GitOps have an opportunity to upskill existing staff for more automation- and code-oriented I&O roles. This allows staff to learn new skills and technologies, increasing employee satisfaction and retention.
- **Cultural change:** By breaking down organizational silos, development and operations leaders can build cross-functional knowledge and collaboration skills across their teams to enable them to work effectively across boundaries.
- **Cost reduction:** Automating infrastructure eliminates manual tasks and rework, improving productivity, which can contribute to cost reduction.
- **Compliance requirements:** The declarative nature of GitOps leaves an easy audit trail for software changes, improving compliance.

Obstacles

- **Prerequisites:** GitOps is only for cloud-native applications. Many GitOps tools and techniques assume the system is built on Kubernetes. By definition, GitOps requires software agents to act as listeners for changes and help to implement them. GitOps is possible outside Kubernetes; however, in practice, K8s will almost certainly be used. Thus, GitOps is necessarily limited in scope.

- **Cultural change:** GitOps requires a cultural change that organizations must invest in. IT leaders must embrace process change. This requires discipline and commitment from all participants to do things differently.
- **Skills gaps:** GitOps requires automation and software development skills, which many I&O teams lack. Practitioners and organizations must be mature enough to operate infrastructure through code. Many are not.
- **Organizational inertia:** GitOps requires collaboration among different teams, which requires mutual trust to be successful.

User Recommendations

- **Target cloud-native workloads initially:** Your first use case for GitOps should be operating a containerized, cloud-native application that is already using both Kubernetes and a continuous delivery platform such as Flux or Argo CD.
- **Build an internal operating platform:** This is the foundation of your GitOps efforts. Your platform should manage the underlying infrastructure and deployment pipelines while enforcing security and policy compliance.
- **Embed security into GitOps workflows:** Security teams must shift left so the organization can build holistic CI/CD pipelines that deliver software and configure infrastructure, with security embedded in every layer.
- **Be wary of vendors trying to sell you GitOps:** GitOps isn't a product you buy. It is a workflow and a mindset shift that becomes part of your DevOps culture. Tools that expressly enable GitOps can be helpful, but GitOps can be done with nothing more than standard continuous delivery tools that support Git-based automation.

Sample Vendors

Akuity; GitLab; Harness; Red Hat; Upbound

Gartner Recommended Reading

[Innovation Insight: Top 4 Use Cases for GitOps](#)

[Use GitOps With Kubernetes to Streamline Deployment and Platform Automation](#)

Designing and Operating DevOps Workflows to Deploy Containerized Applications With Kubernetes

Automate the Application Delivery Value Stream

Developer Experience

Analysis By: Alec Pallin, Brian Minning

Benefit Rating: High

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

Developer experience refers to all aspects of interactions between developers and the tools, platforms, processes and people they work with to develop and deliver software products and services. A superior developer experience requires an environment in which developers can do their best work with minimal friction and maximum flow.

Why This Is Important

Software development teams work in an increasingly complex environment with a growing array of tools, technologies, architectures and processes across the software delivery life cycle. This complexity results in friction and increased cognitive load for developers, limiting their ability to deliver value. Developer experience initiatives holistically address the causes of friction and frustration for teams, enabling them to focus on the highest-value activities with minimal distraction.

Business Impact

A superior developer experience drives a number of key organizational outcomes.

Developers with a high-quality developer experience are more likely to:

- Achieve their target business outcomes, such as revenue growth and user satisfaction.
- Have higher productivity, including better delivery flow, speed to market, release cadence and delivery predictability.
- Have high intent to stay with their employer.

These are summarized findings from the [Gartner Developer Experience Assessment](#).

Drivers

- **Pressure to boost developer productivity:** Software engineering leaders face intensifying pressure to increase and demonstrate their teams' productivity with the emergence of generative AI tools, particularly AI coding assistants. These tools promise significant productivity gains for developers, but to fully realize these gains, leaders must align their selection and use cases with developer experience needs and priorities.
- **Growing complexity of software architectures and development technologies:** As the software engineering tooling and technology landscape continues to grow and evolve, teams experience increasing cognitive load. To deliver a superior developer experience and enable teams to do their best work, leaders must take steps to manage complexity for their teams, such as by providing them with internal developer portals and platforms.
- **Challenges attracting and retaining top talent:** Software engineering leaders continue to report attracting and retaining engineering talent to their organization as key challenges. A high-quality developer experience is an attractive value proposition to potential hires, demonstrating an organizational commitment to providing a productive and engaging work environment.

Obstacles

- **Securing stakeholder buy-in for developer experience investments:** Developer experience initiatives risk underinvestment without a strong business case, because executives often misperceive the purpose and business impact of a good developer experience.
- **Unclear ownership of developer experience initiatives:** With the end-to-end developer journey spanning a complex mix of processes, technologies and stakeholders, organizations without well-defined accountability for managing developer experience initiatives risk piecemeal and overly narrow approaches to addressing needs.
- **Incomplete understanding of developer experience challenges:** The developer experience entails more than just high-quality tools; it also includes other aspects of the end-to-end developer journey such as onboarding, upskilling and workflow design. Organizations that fail to gain a holistic view of development teams' leading pain points risk focusing narrowly on lower-impact developer experience improvements.

User Recommendations

- Treat the developer experience like a product. Assign clear accountability for delivering the end-to-end developer journey to a dedicated developer experience team and product owner, while enabling individual development teams to identify and address local developer experience needs.
- Regularly gather feedback from development teams about their leading challenges and pain points. Employ quantitative and qualitative methods, such as surveys, developer journey-mapping workshops and tool telemetry, and use this data to identify the highest-priority developer experience improvement opportunities.
- Track and measure the impact of developer experience improvements over time by using metrics that capture key delivery, business and development team well-being outcomes.

Sample Vendors

Culture Amp; DX; LinearB; Swarmia

Gartner Recommended Reading

[Ignition Guide to Establishing a Developer Experience Strategy](#)

[Case Study: Holistic Strategy to Deliver a High-Quality Developer Experience \(Hargreaves Lansdown\)](#)

[Developer Experience Assessment](#)

[How to Measure Developer Experience](#)

AI-Augmented Software Engineering

Analysis By: Arun Batchu, Manjunath Bhat, Nitish Tyagi, Keith Holloway

Benefit Rating: Transformational

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

AI-augmented software engineering (AIASE) refers to the integration of AI as a collaborative partner to enhance the capabilities of software engineers across the software development life cycle (SDLC). It aims to accelerate the delivery of high-quality software by leveraging AI technologies in various phases, including planning, design, development, testing, validation, security, deployment and maintaining applications.

Why This Is Important

The SDLC encompasses routine tasks within both the creative and operational DevSecOps loops. AI automation reduces manual effort, allowing engineers to focus on innovation, reduce technical debt, enhance quality and security, improve team collaboration, and lower operational costs. AI technologies augment engineers' cognitive tasks, including analyzing logs, optimizing configurations and generating scripts, code, unit tests and documentation.

Business Impact

AIASE transforms the SDLC by integrating AI as a collaborative partner. This integration accelerates the delivery of high-quality software and enhances various phases of development, including planning, design, testing and maintenance. By automating routine tasks and augmenting cognitive tasks such as log analysis and code generation, AIASE reduces manual effort, allowing engineers to focus on innovation. This leads to reduced technical debt, improved quality and security, enhanced team collaboration and lower operational costs, ultimately providing transformational benefits to organizations.

Drivers

Demand-side drivers include:

- Increased complexity of software systems to be engineered
- Increased demand for developers to deliver high-quality code faster
- Increased number of quality-related risks associated with software development
- The need to protect against increasingly sophisticated security threats
- The need to optimize operational costs

Supply-side drivers:

- Applying AI models to improve software quality by detecting defects, and fixing them

- Increasing impact of software development on business
- Applying foundation models such as large language models (LLMs) to software code generation and optimization
- Applying deep learning models to software operations

Obstacles

- The 2024 Gartner Technical Architect Survey highlights a growing skills gap in AI/ML, with 31% reporting high demand but persistent shortages across organizations. Top-performing firms increasingly seek software developers with data science skills. And the 2025 Gartner Software Engineering Survey identifies prompt engineering as an emerging skill, underscoring the rising importance and challenge of addressing these skill shortages.
- Employees fear that job automation by AI will lead to job elimination.
- The hype surrounding this innovation has led to misconceptions and unrealistic expectations regarding AIASE's advantages.
- Myopic focus on code generation has reduced opportunities for AI tool application throughout the SDLC.
- Solutions are uneven and fragmented, automating only certain tasks within the SDLC.
- Intellectual property risks and privacy issues arise from AI models trained on restrictive licensed code and proprietary or leaked data.

User Recommendations

- Pilot tools to assess potential gains, measure their effectiveness, and then roll out or scale them broadly if the results are positive.
- Innersource best practices, featuring automated prompt generation through saved examples, to effectively use AIASE technology.
- Validate the accuracy and verify the maintainability of AI-generated artifacts via human or automated reviews.

- Enter only short-term contracts with vendors to ensure the ability to migrate to more effective tools as the market matures.
- Ensure psychological safety for software engineers by emphasizing that AIASE serves as a learning partner and augmentation toolset, supporting rather than replacing human expertise, and offering continued career development opportunities.
- Establish guidelines to choose providers that offer transparency in training data and model processes. Prioritize indemnified commercial models for critical applications, and use tools like ScanOSS for licensing compliance.
- Establish the correct set of metrics, such as new release frequency and quality artifacts, to measure AIASE's success.

Sample Vendors

Amazon Web Services; Anima; Atlassian; CAST; Codeium; Dynatrace; GitHub; GitLab; Sedai; Veracode

Gartner Recommended Reading

[How Platform Engineering Teams Can Augment DevOps With AI](#)

[Market Guide for AI-Augmented Software-Testing Tools](#)

[Magic Quadrant for AI Code Assistants](#)

[Innovation Insight for AI-Native Software Engineering](#)

[Critical Capabilities for AI Code Assistants](#)

Design Systems

Analysis By: Will Grant, Brent Stewart

Benefit Rating: High

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

Design systems are collections of reusable assets that are based on clear visual, user interface and technical standards. They serve as building blocks to quickly and consistently design and develop digital products. Organizations can deliver better experiences for customers and reduce development effort, while preparing to leverage emerging generative AI (GenAI) technologies. A complete design system comprises style assets, structural assets, code components and documentation.

Why This Is Important

Using a design system is one of the most effective ways of ensuring visual and user experience (UX) consistency across digital product offerings. Building a design system into your software development process contributes to increased brand consistency, accessibility, better UX and higher front-end developer productivity. Popular SaaS platforms — including Salesforce and SAP — maintain their own design systems to facilitate application design and development on their platforms.

Business Impact

A design system is one of the most important strategic assets for an organization that builds and configures digital products. A robust design system that is well-resourced and maintained will:

- Shorten design and development timelines
- Improve the user interface (UI) design
- Ensure that UX is consistent and predictable
- Guarantee brand compliance across an organization's full portfolio of digital products, both customer- and employee-facing

Drivers

- **Speed:** Design systems reduce the time required to design and code front-end software by minimizing the need to repeatedly design from a blank template. Design systems enable easy component assembly and fast screen-design tweaks that allow designers to work at pace.
- **Usability:** Design systems are typically composed of proven UI design patterns that are familiar to most users. Foundational UX design heuristics, such as “visibility of system

status” or “recognition rather than recall,” are built into these patterns.

- **Consistency:** Design systems enable the creation of consistent UX across disparate teams. These can be feature teams for a single product, or multiple product teams sharing a design system across a larger product portfolio and other modalities like in-store kiosks and wearables.
- **Scale:** Design systems make it easy for designers and developers to share common, approved design assets and code components across an entire portfolio of digital products, and to work independently using the same assets.
- **Improved product support:** Software tooling is increasingly supporting design systems as a “first class” feature, for example in Figma, Penpot and Storybook.
- **Reduction of defects:** Over time, design system code components become “hardened,” leading to far fewer production defects in the presentation layer.
- **Brand compliance:** Design systems reinforce a brand identity and infuse key elements such as color and typography into every single design and code asset.
- **Accessibility:** Design system assets can be created in compliance with the latest Web Content Accessibility Guidelines (WCAG), eliminating unnecessary rework downstream and setting teams up to deliver front ends that meet accessibility legislation.
- **AI-ready:** A codified design system will be essential to enable GenAI to produce screen designs or front-end code that is consistent with the established product look and feel.

Obstacles

- **Effort to create and maintain:** While design systems bring many benefits, they need to be promoted to gain adoption, and resourced and maintained like any other internal product in order to deliver sustained value over time.
- **Integration challenges:** Design systems often need to be integrated into existing legacy systems, which can be technically challenging and resource-intensive.
- **Cross-discipline buy-in:** Without the whole software engineering team getting behind a design system, there’s a risk that several ad hoc design systems will emerge, multiplying effort and reducing impact.

- **Executive buy-in:** Few executive leaders outside of the design field are aware of the strategic importance and tremendous business value of design systems. Without strong leadership support, design systems easily become underutilized and diminished in terms of the value they add.

User Recommendations

- Conduct a regular review of available design systems by auditing leading open-source examples.
- Assemble a team including UX, product development and product marketing to gather, organize, define and launch an enterprisewide design system. Treat the design system like an internal product, with a backlog, roadmap and team.
- Avoid starting a custom or proprietary design system from scratch unless you're prepared for a significant multiyear investment to catch up with established systems.
- Update design and development processes to mandate the use of the design system rather than beginning again for new initiatives.
- Document your design system with style guides, technical component documentation, usage guides and accessibility considerations.
- Consider design system platforms (such as Storybook, zeroheight and Knapsack) that support design tokens and reusable components to enable the operationalization of your design system and its readiness for AI-augmented design tools in the future.

Sample Vendors

Ant Group; Google; IBM; Microsoft; Tailwind CSS

Gartner Recommended Reading

[Start Using Design Systems, Accelerate Digital Product Delivery](#)

[Build Better Products Faster With Design Systems](#)

[How Generative AI Is Changing User Experience](#)

[The 4 Secrets of Design-Led Companies](#)

Value Stream Management Platform

Analysis By: Hassan Ennaciri, Akis Sklavounakis

Benefit Rating: High

Market Penetration: 1% to 5% of target audience

Maturity: Emerging

Definition:

A value stream management platform (VSMP) optimizes end-to-end product delivery and improves business outcomes by connecting to existing tools and ingesting data from all phases of software product delivery (from customer needs to value delivery). These platforms, typically data-source agnostic, help software engineering leaders identify and quantify opportunities to improve software product performance by optimizing cost, operating models, technology and processes.

Why This Is Important

As organizations scale their product delivery, the need for metrics and insights that assess delivery performance and product outcomes is expanding. VSMPs integrate with multiple data sources to provide delivery telemetry. These insights enable stakeholders to make data-driven decisions in an agile manner and correct course as needed. The visualization capabilities of VSMPs help product teams analyze customer value metrics against the cost required to deliver that value.

Business Impact

VSMPs can provide CxOs and IT leaders with strategic views of product delivery, and measure value output, health and performance, allowing them to make data-driven decisions about future product investments. These platforms also provide product teams with end-to-end visibility and insight into the flow of work to help them address constraints and improve delivery.

Drivers

- Companies seek to improve their business results through better software.
- Need to optimize product delivery and align with business priorities and objectives.

- Timely decision making on product development priorities driven by insights from data.
- Optimization of delivery flow through reduction of waste and elimination of bottlenecks.
- Visibility and mapping of end-to-end software delivery processes and identification of cross-team dependencies.
- More stringent governance, security and compliance requirements.
- VSMPs help organizations bridge the gap between business and IT by enabling stakeholders to align their priorities to focus on delivering customer value.

Obstacles

- VSMP is an additional investment to complement continuous integration/continuous delivery (CI/CD) capabilities. They do not replace DevOps platforms or custom toolchains.
- VSMPs rely on data from various tools. Integrating these tools and ensuring data compatibility can be complex and time-consuming.
- There are increasing capabilities from software engineering intelligence products and features of DevOps platforms that are making investments into VSMPs more difficult to justify.
- VSMPs require customization and data from tools used by multiple stakeholders in the organization, sometimes outside of software delivery. Collaboration with these key stakeholders to deliver the desired insights is paramount.
- Without well-defined goals and objectives, it can be challenging to measure the success of a VSMP implementation and demonstrate its value.

User Recommendations

- Identify the most critical value streams in your organization. Begin by implementing the VSMP for these key areas to demonstrate value and build momentum.
- Carefully plan how you will integrate the VSMP with your existing tools (for example, DevOps platforms, Jira, CI/CD pipelines, testing tools).
- VSMP implementation requires collaboration across all teams involved in the product value stream. Encourage open communication and ensure everyone understands the

goals and benefits of the VSMP.

- Use VSMPs' AI-powered analytics and insights to surface constraints, detect bottlenecks and improve flow.
- Build customized dashboards and views of product delivery and value for multiple stakeholders and leadership.
- Utilize VSMPs to assess the performance, quality and value of products, including development costs and ROI.
- Use VSMPs to gain a consolidated view of governance, security and compliance across all product lines.

Sample Vendors

Broadcom; ConnectALL; Digital.ai; HCLSoftware; IBM; OpenText; Opsera; Planview; ServiceNow

Gartner Recommended Reading

[Market Guide for Value Stream Management Platforms](#)

[Magic Quadrant for DevOps Platforms](#)

[Essential Software Metrics: DORA](#)

[Is DORA, SPACE or DevEx the Right Software Metrics Framework?](#)

[Market Guide for Developer Productivity Insight Platforms](#)

Sliding into the Trough

Value-Based Metrics

Analysis By: Mukul Saha

Benefit Rating: High

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

Amid an increasingly VUCA (volatile, uncertain, complex and ambiguous) world, value-based metrics are vital in helping agile teams stay focused on delivering measurable outcomes. Metrics such as quality, lead time, cycle time, velocity and team happiness must align with business goals like improved customer satisfaction. Value-based metrics enable data-driven decisions and continuous improvement, fostering resilience and ensuring alignment in volatile, fast-changing environments.

Why This Is Important

As software engineering leaders transform their organizations to adopt agile development and DevOps approaches, they struggle to find the right metrics to demonstrate business value. Metrics are essential for communicating and aligning engineering performance with business value.

Business Impact

Value metrics enable organizations to optimize their processes and improve productivity by identifying bottlenecks and making data-driven improvements. This leads to more efficient product delivery and resource allocation by working on the things most important to the business. More than ever, metrics such as customer satisfaction, revenue growth and time to market provide insights into the incremental value delivered by the agile approach.

Drivers

The adoption of value-driven metrics is based on the expectation that it will deliver the following benefits:

- **Strategic business value:** Metrics like return on investment (ROI) or business value delivered measure engineering's impact on the organization's bottom line.
- **Customer satisfaction:** Agile methodologies prioritize customer satisfaction by delivering value to customers quickly and continuously. Metrics such as Net Promoter Score (NPS) or customer satisfaction surveys can measure customer satisfaction.
- **Time to market:** Cycle time or lead time can measure the time it takes to produce a new feature or product for the market.

- **Product quality:** Defect density or customer-reported defects can measure the quality of the product.
- **Team productivity:** Velocity or throughput can measure the work the team completes in a given time frame. Teams are more productive if they feel connected to their work and find it meaningful.
- **Employee satisfaction:** Employee satisfaction or team morale surveys can measure the level of satisfaction and engagement of team members.
- **Adaptability and flexibility:** Change success rate or adaptability index can measure the team's ability to respond to change.

Obstacles

- Lack of stakeholder understanding and support hampers the adoption of value metrics.
- Resistance to change impedes the integration of value metrics.
- Inconsistent use of tools and processes and infrastructure hinder the effective use of value metrics.
- Unreliable or insufficient data poses challenges to leveraging value metrics.
- Overemphasis on individual output metrics undermines the comprehensive use of value metrics.
- Difficulties in defining value complicate the implementation of value metrics.
- Lack of transparency and collaboration obstructs the successful utilization of value metrics.
- Unclear enterprise objectives and definition of success in SMART (specific, measurable, achievable, relevant and time-bound) terms. Once a SMART goal is achieved, teams should focus on the next important enterprise objective.

User Recommendations

- Ensure that all stakeholders, including software engineering team members, understand why value metrics are being implemented and how they contribute to the organization's overall goals.

- Provide training and workshops to help stakeholders, including team members and management, understand the value metrics.
- Set up feedback mechanisms to collect accurate and reliable data for the chosen metrics.
- Encourage open communication and trust among team members and stakeholders.
- Create a community of practice where sharing data, insights and lessons learned is encouraged, leading to better collaboration and continuous improvement.
- Celebrate achievements with a selected small set of meaningful metrics and actively socialize them to ensure shared understanding and focus. Similarly, treat failures as learning opportunities that fuel continuous improvement.

Sample Vendors

Atlassian; Digital.ai; Microsoft; Planview; ServiceNow

Gartner Recommended Reading

Essential Metrics for Agile and DevOps Teams

Reframe Software Engineering Metrics for Agile and DevOps Success

Is DORA, SPACE or DevEx the Right Software Metrics Framework?

How to Measure and Improve Developer Productivity

Tool: Framework to Link Distributed Delivery Team's Operational Metrics to Business Outcomes

AI-Augmented Testing

Analysis By: Joachim Herschmann, Jim Scheibmeir

Benefit Rating: High

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

AI-augmented testing comprises AI- and machine learning (ML)-based technologies and practices to make software testing activities more independent from human intervention. It continuously improves testing outcomes by learning from the data collected from performed activities. It extends traditional test automation beyond the automated execution of test cases to include automated planning, creation, maintenance and analysis of tests and test prioritization, test analysis and test value scoring.

Why This Is Important

Software engineering leaders seeking to release faster without degrading quality are looking for more efficient ways of testing across all phases of the software life cycle. AI-augmented testing enables the automation of a broad set of testing activities related to the quality of requirements, design quality, code quality, release quality and operational resilience. This increases the degree of autonomy of those activities.

Business Impact

The adoption of AI-augmented testing has the potential to democratize testing and significantly improve an IT organization's ability to serve and delight its customers. It can enable the fine-tuning of scenarios for testing as part of a continuous quality strategy aimed at optimizing the end-user experience. It will also help to build a closed-loop system that quickly provides continuous feedback about critical quality indicators and helps to reduce the costs of creating and maintaining tests.

Drivers

- A high dependency on human expertise and interaction limits how quickly modern digital businesses can design, build and test new software.
- Where automated testing is already in place, current levels of automation often remain below expectations due to a continued dependency on human intervention to maintain the automation as applications under test (AUT) evolve.
- The pressure to innovate quickly for market differentiation without compromising on quality relies on both an increased velocity and a higher degree of autonomy of the related activities.
- Product teams struggle to deal with the increasing complexity of applications, leading to increased cognitive overload. Complex architectures increasingly require an

understanding of an array of elements including cloud-native architecture, replacement of technologies, use of microservices, and support for multiple frontends and AI-powered services.

- Increased adoption of agile and DevOps results in a faster development and delivery cadence, but also comes with additional responsibilities.
- Existing work backlogs to replace manual tests with automated tests that support continuous delivery of software are ever-increasing.
- There is a shortage of skilled test automation engineers to close these automation backlogs.
- Businesses want to reduce test operation and maintenance costs associated with traditional tools and open-source software (OSS) solutions.
- Businesses are aiming to improve the user experience of testing tools so quality engineers can be more productive and avoid mistakes.
- Compliance regulations such as General Data Protection Regulation (GDPR) for data privacy and Web Content Accessibility Guidelines (WCAG) 2.1 Level AA for accessibility are enhanced by AI-augmented testing.

Obstacles

- Currently available tools are still relatively new, have a narrow scope and still need to prove their value when used at scale. Large language model (LLM)-based generative AI (GenAI), in particular, exhibits hallucinations (content that is nonsensical or untruthful in relation to certain sources), a problem that will not go away completely because of their probabilistic nature. In addition, potential copyright violations and security issues remain risks associated with LLM-based AI technologies.
- Risks, common to GenAI, are exacerbated when AI agents gain agency and handle more complex operations autonomously. Left unchecked, agents can execute processes that compromise the system, violate regulations or expose data to unauthorized parties.
- Underestimating the time required to acquire new skills and setting wrong expectations about the time required to become successful can be obstacles.

User Recommendations

- Set the right expectations about the potential and limitations of AI-augmented testing and ensure that humans are always in the loop to verify the results produced by AI-augmented testing tools. This is particularly relevant for tools employing GenAI to automatically create tests, as generated tests may be completely useless or result in false positives or negatives.
- Start evaluating AI-augmented testing tools now to understand the current possibilities and limitations of these products. Build a roadmap to solve the development organization's most pressing quality challenges.
- Increase the value of AI-augmented testing tools by exploring additional use cases beyond core test automation scenarios, which limit automation primarily on the execution of tests. For example, look for shift-left scenarios such as generating test scenarios from requirements or from user stories and contextual information contained within the codebase (including code and documentation).

Sample Vendors

Applitools; Katalon; Keysight; OpenText; Parasoft; Perforce; SmartBear; Tricentis; UiPath

Gartner Recommended Reading

Market Guide for AI-Augmented Software-Testing Tools

Quick Answer: How Can AI Provide Benefits for Software Testing?

Innovation Insight: Continuous Quality

Improve Software Quality by Building Digital Immunity

AI Use-Case Comparison for Software Development and Testing

Observability

Analysis By: Padraig Byrne, Gregg Siegfried

Benefit Rating: Transformational

Market Penetration: 20% to 50% of target audience

Maturity: Adolescent

Definition:

Observability is the characteristic of software and systems that enables them to be understood based on their outputs and enables questions about their behavior to be answered. Platforms that facilitate software observability enable observers to collect and explore high-cardinality telemetry using techniques that iteratively narrow the possible explanations for errant behavior.

Why This Is Important

The complexity of modern applications and the rise of practices such as DevOps has left organizations frustrated with legacy monitoring tools and techniques. These can do no more than collect and display external signals, which results in monitoring that is, in effect, only reactive. Observability acts like the central nervous system of a digital enterprise.

Observability platforms surface data that allows system behavior to be better understood and anomalies to be detected and triaged.

Business Impact

Observability platforms have the potential to reduce both the number of service outages and their severity. Their use by organizations can improve the quality of software because previously invisible (unknown) defects and anomalies can be identified and corrected before service is degraded. By enabling stakeholders to better understand how their products are used, observability also supports the development of more accurate and usable software.

Drivers

- The term “observability” is now ubiquitous, with uses extending beyond the domain of IT operations. Care must be taken to ensure the term retains relevance when used beyond its original range of reference.
- OpenTelemetry’s progress and continued acceptance as the “observability framework for cloud-native software” raises observability and its toolchain.
- Traditional monitoring systems capture and examine signals in relative isolation, with alerts tied to threshold or rate-of-change violations that require prior awareness of possible issues and corresponding instrumentation. Given the complexity of modern applications, it is unfeasible to rely on traditional monitoring alone.

- Observability platforms enable an observer, a software developer or a site reliability engineer to explain and predict unexpected system behavior more effectively, provided enough instrumentation is available. Integration of observability with AI applications to automate subsequent actions is now becoming a reality for many observability platform vendors.
- Observability is an evolution of longstanding technologies and methods, and established monitoring vendors are well on their way to incorporating observability into their products. New companies are also creating innovative observability offerings at a fast rate.
- Expectations of IT services have evolved, driving enterprise software vendors to consider user experience as a key component of application success.

Obstacles

- In many enterprises, the role of IT operations has been to “keep the lights on,” despite constant change. This, combined with the longevity of existing monitoring tools, means that adoption of new technology is often slow.
- Enterprises have invested significant resources in their existing monitoring tools, which exhibit a high degree of “stickiness.” This creates organizational barriers to adopting new practices, such as those related to observability.
- Costs associated with observability platforms have grown as companies struggle to keep up with the explosion in volume and velocity of telemetry.
- Lack of skilled resources and a steep learning curve continue to be a constraint in implementing observability, particularly for less familiar techniques such as distributed tracing and continuous profiling.

User Recommendations

- Assess observability platforms’ ability to integrate into continuous integration/continuous delivery pipelines and feedback loops.
- Investigate problems that cannot be framed by traditional monitoring by using observability to add flexibility to incident investigations.

- Encourage observability practices by focusing on vendors that make it easy to use open standards for collection, such as OpenTelemetry.
- Tie service-level objectives to desired business outcomes using specific metrics and use observability platforms to understand variations.
- Ensure IT operations and site reliability engineering teams are aware of updates to existing observability platforms and how they may take advantage of them. Many traditional application performance monitoring vendors are starting to incorporate observability features into their products.
- Understand that even with observability, some services are still not possible to instrument, particularly third-party SaaS services. Organizations should use alternative methods, such as digital experience monitoring, to close such visibility gaps.

Sample Vendors

Chronosphere; Coralogix; Dynatrace; Grafana Labs; Honeycomb; Observe

Gartner Recommended Reading

Magic Quadrant for Observability Platforms

Critical Capabilities for Observability Platforms

Site Reliability Engineering

Analysis By: George Spafford, Daniel Betts

Benefit Rating: Transformational

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

Site reliability engineering (SRE) is a collection of systems and software engineering principles used to design and operate scalable resilient systems. Site reliability engineers collaborate with customers or product owners, using operational data and feedback, to define service-level indicators (SLIs) and service-level objectives (SLOs). Site reliability

engineers work with product or platform teams to design, operate and continuously optimize systems that meet defined SLOs.

Why This Is Important

SRE has evolved reactive and proactive engineering practices that enable customer-focused reliability while supporting effective and efficient scaling. Individual organizations implement SRE in widely varying ways, depending on their goals. SRE teams can work with product and platform teams to help them achieve the levels of reliability that the product and platform owner are accountable for.

Business Impact

The SRE approach to improving reliability is intended for critical products and platforms that need to deliver customer value at speed and at scale while managing risk. Within each product or platform, the critical customer journeys, processes or transactions must be addressed also. The exact impact depends on the business driver impacted, for example, time to market, revenue and customer satisfaction.

Drivers

- Organizations are under pressure to meet customer requirements for reliability while scaling their digital services and are looking for guidance to help them.
- SRE has evolved since its initial implementations. Reliability remains the focus and integrates a rich body of knowledge that complements agile, DevOps and platform engineering approaches.
- Organizations that have adopted highly skilled automation practices (usually DevOps) and usage of infrastructure-as-code capabilities to deliver digital business products expand to incorporate reliability into these products.
- The most common use case, judging from inquiry calls with Gartner clients, is to leverage SRE concepts to improve the reliability of existing systems that are not meeting customer requirements for availability or performance or proving difficult to scale.

Obstacles

- Belief that reliability is an I&O responsibility only, rather than being a shared organizational goal.

- Organizations have difficulty shifting mindsets from service-level agreements with blanket availability statements to defining and measuring reliability in terms of customer-focused SLOs.
- Defining appropriate SLOs that reflect both business needs and user expectations can be complex.
- Finding SRE role candidates with the right mix of development, operations and people skills is a challenge for clients. This impacts initial adoption and scaling efforts.
- Rebranding of a traditional operations team without changing to adopt SRE practices leaves teams as SRE in name only, without the benefits.
- Clients have voiced problems with product owners who overly focus on functional requirements and not nonfunctional requirements, thus hampering improvements.
- SRE requires close collaboration between development and operations teams. Organizational silos can make it difficult to implement SRE effectively.
- SRE initiatives require executive sponsorship and investment to effectively scale.

User Recommendations

- Define your goals for SRE.
- Begin your SRE journey with a small focused initiative and then iteratively grow adoption. Demonstrate value to gain organizational commitment.
- Select opportunities that are politically friendly, will demonstrate sufficient value and have an acceptable risk profile.
- Automate for SRE efficiency. Identify repetitive tasks and automate them. This frees up engineers to focus on more strategic work.
- Work with the product owner to identify the SLIs and SLOs necessary to make customers and users happy — not to perfection.
- Implement monitoring and improve observability to objectively report on actual performance relative to error budgets and SLOs.
- Ensure product owners treat SLOs as a feature and are accountable for them.

- Instill collaboration among site reliability engineers, developers and other stakeholders.
- Create a community and leverage organizational learning practices while evolving SRE practices.
- Cultivate a blameless culture where mistakes are seen as opportunities for learning and improvement.

Sample Vendors

Datadog; FireHydrant (Blameless); Harness; New Relic; Nobl9; Sedai; ServiceNow; Splunk

Gartner Recommended Reading

[7 Steps to Start and Evolve Your SRE Practice](#)

[Improve Customer Experience and Reliability With SLIs and SLOs](#)

[Tool: Sample Job Description for Site Reliability Engineer \(SRE\)](#)

[Market Guide for Site Reliability Engineering Tooling](#)

Agile Industrial Complex

Analysis By: Peter Hyde, Nabeeha Ahmed

Benefit Rating: Low

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

The agile industrial complex critiques the institutionalization and commercialization of agile in software engineering. It refers to how agile has become a global industry of certifications, training, consulting services and inflexible frameworks that prioritize profit over agility.

Why This Is Important

The agile industrial complex critique is crucial as it underscores the challenges agile faces. Many organizations perceive agile as hard to implement and tough to scale, ultimately failing at its promised value. The commercialization by agile training companies emphasizing

certifications and consulting services obstructs organizations from attaining business agility and optimizing value creation. This critique encourages a reevaluation of agile practices, emphasizing innovation and adaptability.

Business Impact

The overcommercialization of agile has resulted in rigid frameworks and standardized processes, with an excessive focus on certifications and tools. These inflexible, proprietary frameworks and the emphasis on consulting services stifle innovation and adaptability, inflate transformation costs and cause skepticism and disillusionment. This shift diverts attention from agile's core values and principles, undermining its potential to drive genuine business agility and value creation.

Drivers

- **Overcommercialization:** The proliferation of proprietary agile frameworks, certifications, training programs and consulting services prioritizes profit over foundational values and principles, resulting in inflexible and inefficient implementation that undermines true agility.
- **Mission drift:** Since the Agile Manifesto's inception in 2001, its original intent has faded amid a landscape of competing businesses, each offering divergent interpretations, diluting its core message and purpose.
- **Agile misapplication:** Practices are often misunderstood or misapplied, resulting in ineffective implementations that fail to deliver promised benefits. Rigid processes and incompatible tools reduce optionality, impede flexibility, harm collaboration and detract from a customer-focused approach.

Obstacles

- **Commercial self-interest:** Entities behind agile frameworks and training programs frequently resist developing open standards or engaging with agile meta frameworks, prioritizing proprietary interests over collaborative advancement.
- **Sunk cost fallacy:** Extensive investments in established agile frameworks deter training organizations from adapting to negative feedback and evolving market dynamics, hindering responsiveness and innovation.

- **Traditional mindset:** Past performance is no guarantee of future results. Reliance on past successes fails to address the distinct needs of modern product teams. Resistance to experimentation and flexibility in working methods obstruct ongoing modernization efforts.

User Recommendations

- **Reinforce core values:** Revisit and discuss agile's foundational values and principles to ensure collaboration, flexibility and customer centricity are at the heart of your daily practices and decision making. Ensure your agile practices are working for you and not simply reflecting how work used to be done.
- **Prioritize learning:** Promote continuous learning and experimentation over formal training and certifications. Use communities of practice for peer-to-peer knowledge sharing and support.
- **Customize practices:** Tailor agile methodologies to meet the specific needs of your teams and products, fostering innovation and responsiveness. Review using an agile metaframework to blend agile practices into a cohesive way of working.
- **Develop internal expertise:** Invest in cultivating agile knowledge in your organization by empowering team members to take on coaching roles and lead agile workshops, minimizing reliance on external consulting services.

Gartner Recommended Reading

[The Why, What and How of Successful Agile Transformation](#)

[14 Ways Your Agile Adoption Will Fail](#)

Software Supply Chain Security

Analysis By: Manjunath Bhat

Benefit Rating: Transformational

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

Software supply chain security (SSCS) is about building secure software by protecting against compromised code, tooling, identities and pipelines during development, delivery and postdeployment. SSCS reduces third-party risks through policy-based curation of dependencies, software composition analysis (SCA) and software bill of materials (SBOM) inspection. They ensure artifact provenance and traceability with signing and verification as they pass through development and delivery pipelines.

Why This Is Important

Software supply chains transcend organizational boundaries and consist of external entities in addition to internal systems. Internal systems include software delivery pipelines, software dependencies and software development environments. External entities include partners, open-source software (OSS) and vendors. Organizations have greater control over internal systems and little to no control over external entities. SSCS protects organizations both from insider threats as well as external entities that are potentially exploitable.

Business Impact

- Identifies and mitigates security and compliance risks arising from the widespread use of open-source software.
- Reduces developer friction and lost productivity due to attacks on tools, environments, pipelines and infrastructure used for software development, delivery and runtimes.
- Satisfies governance and regulatory requirements by making the software delivery infrastructure auditable by automating enforcement of application security policies.

Drivers

- **State-sponsored attacks:** Much of open-source software is vulnerable to infiltration by nation-state bad actors. State-sponsored software supply chain attacks have, therefore, become increasingly sophisticated and prevalent, targeting various sectors globally. These attacks have either used vulnerabilities in the software itself (as in the case of Kaseya) or compromised the software development life cycle (SDLC) (as in the cases of SolarWinds and NotPetya attacks). SSCS becomes critical to reduce these risks.
- **Regulatory compliance and government mandates:** Governments, policymakers and regulatory bodies around the world are mandating third-party supplier assessments, continuous vulnerability scanning and software bills of materials to build a trusted

software supply chain. Examples of mandatory regulations include the **United States Government Cybersecurity Executive Order 14028**, **EU Cyber Resilience Act**, **EU NIS2 Directive**, and the **Federal Food, Drug, and Cosmetic (FD&C) Act**.

- **Pervasive use of open source and reliance on third-party software:** Every software application uses third-party code by way of open-source dependencies. Based on hundreds of analyst-client interactions, Gartner believes that more than 95% of organizations are using OSS, even if they are not aware of it. According to the 2025 BlackDuck Open Source Security and Risk Analysis Report, 97% of the codebases contained open source. The same report finds 86% of audited applications contained open-source vulnerabilities, with 81% of the applications containing high- or critical-risk vulnerabilities.
- **Use of open-weight AI models:** The ease of access to open-weight large language models (LLMs) and the low barrier to integrating them in applications add a new dimension to software supply chain risks. The Gartner Software Engineering Survey for 2025 shows 47% of respondents integrating LLMs into existing applications. SSCS helps to discover use of LLMs, scan models for risks and enforce policies to prevent the use of unapproved LLMs. Examples of risks include weak model provenance, unsupported models and geopolitical barriers preventing model use.

Obstacles

- The majority of organizations lack a complete understanding of SSCS, and thus have yet to adopt a comprehensive approach to manage software supply chain risks. For example, many organizations are focused on acquiring SBOMs but have yet to establish how those artifacts will be evaluated, stored and used.
- Efforts to secure the integrity and provenance of software artifacts throughout the software supply chain are emerging but are uneven in scope, execution and adoption. Implementing policies that determine what dependencies are allowed can introduce friction and require negotiation of processes with affected stakeholders (software engineering, application security and platform engineering).
- The adoption of capabilities to harden DevOps pipelines through artifact integrity validation and automated policy enforcement are comparatively lower. This is because of the historically poor developer experience associated with signing and verification workflows in continuous integration/continuous delivery (CI/CD) pipelines. The

heterogeneity of tools in the DevOps pipeline exacerbates the challenge of creating artifact attestations and ensuring pipeline integrity.

User Recommendations

- Enhance trust in your software supply chains by improving visibility, protecting integrity and enhancing security posture throughout the SDLC.
- Reduce visibility gaps in the software supply chain by using software composition analysis and software bills of materials to manage third-party risks as well as ensuring auditability and traceability across pipeline activities and interactions in the SDLC.
- Protect software integrity throughout the software delivery process by signing and verifying build artifacts, ensuring provenance data is in place and preventing use of noncompliant artifacts.
- Improve the security posture of the software delivery process by automating policy enforcement in the SDLC as well as detecting and resolving misconfiguration errors in DevOps tooling.

Sample Vendors

ActiveState; Anchore; Apiiro; Aqua Security Software; Arnica; BlueFlag Security; Boost Security; Cyscale; Endor Labs; GitHub

Gartner Recommended Reading

[Market Guide for Software Supply Chain Security](#)

[Mitigate Enterprise Software Supply Chain Security Risks](#)

[How Software Engineering Leaders Can Mitigate Software Supply Chain Security Risks](#)

[Innovation Insight for SBOMs](#)

DevOps Platforms

Analysis By: Keith Mann

Benefit Rating: High

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

DevOps platforms provide fully integrated capabilities to enable continuous delivery of digital products using agile and DevOps practices. Their capabilities span the development and delivery life cycle built around the continuous integration/continuous delivery (CI/CD) pipeline and include aspects such as versioning, testing, security, documentation and compliance. DevOps platforms support team collaboration, consistency, tool simplification and measurement of delivery metrics.

Why This Is Important

Organizations use DevOps platforms to minimize tool friction and operational complexity resulting from disparate toolchains, manual handoffs and lack of consistent visibility throughout the digital product life cycle. This enables product teams to deliver customer value faster without compromising quality. The DevOps platforms market reflects the consolidation of technologies across development, security, infrastructure and operations to streamline digital product delivery.

Business Impact

DevOps platforms enable continuous delivery of business value. The seamless integration, automation, extensibility and shared visibility between development, security and operations workflows help bridge the silos that exist between these teams. Using a common platform for development, security and operations accelerates agile transformation and helps organizations in transitioning to a product and platform team operating model.

Drivers

- **Modernizing application architectures:** Modernizing applications to take advantage of emerging AI technologies and cloud-native architectures requires fundamental changes to underlying DevOps practices and tools.
- **Enhancing developer productivity:** Increased attention and focus are being placed on improved developer experience, agility, and the need to improve delivery cadence by reducing cognitive load caused by continuous context switching and repetitive low-value tasks.

- **Integrated approach to security and compliance:** Integrating and automating security, compliance and governance as part of the development and delivery process is becoming a priority. This demands maintaining productionlike development and testing environments, as well as managing the software supply chain.
- **Improved visibility into the flow of work:** Organizations are under pressure to reduce friction and manual handoffs. This requires complete visibility into software delivery pipelines from ideation to production.
- **Infrastructure and database operations automation:** While DevOps initially focused on the delivery of software and the automation of related processes, infrastructure and databases are key parts of digital products and their operations can be optimized through automation.

Obstacles

- Organizations that want to unlock the full benefits of DevOps platforms must be willing to replace existing custom toolchains, either entirely or partially. Teams may perceive the change as a disruption to their established ways of working and they may resist any change to the tools they have been using.
- A mature platform engineering capability is required to gain the full benefits of DevOps platforms, but many organizations lack such a capability.
- Organizations accrue technical and skills debt over time due to outmoded automation workflows and legacy applications. This hinders teams from adopting new tools.
- Dependency on a single provider for a majority of their software development needs increases concentration risk and lowers bargaining leverage.
- Most DevOps platforms currently fall short in providing the full set of software delivery capabilities that organizations require to build, deliver, measure and improve the flow of value in the software delivery life cycle.

User Recommendations

- Adopt agile methods and practices to fully reap the business benefits of DevOps platforms.

- Scale and deliver capability by providing DevOps platforms as self-service platforms to reduce overhead, lower complexity, and ensure consistent and templated environments and workflows across multiple teams.
- Improve the flow of value by streamlining the software delivery life cycle with DevOps platforms that provide enhanced visibility across the DevOps pipeline.
- Reduce inconsistency in CI/CD pipeline definitions between teams by utilizing declarative and shareable pipeline capabilities in DevOps platforms.
- Integrate relevant regulatory compliance and information security controls into the DevOps platform, including evidentiary data collection.
- Manage the software supply chain through the use of software bills of materials and related practices.

Sample Vendors

Atlassian; Buildkite; CircleCI; CloudBees; GitHub; GitLab; Harness; Huawei; JetBrains; Octopus

Gartner Recommended Reading

[Keys to DevOps Success](#)

[Research Roundup for DevOps](#)

Chaos Engineering

Analysis By: Jim Scheibmeir, Hassan Ennaciri

Benefit Rating: Moderate

Market Penetration: 20% to 50% of target audience

Maturity: Adolescent

Definition:

Chaos engineering (CE) is the use of experimental and potentially destructive failure testing or fault injection to uncover vulnerabilities and weaknesses within a distributed system.

Chaos engineering tools provide the ability to systematically plan, document, execute and analyze an attack on components and whole systems throughout a system's life cycle.

Why This Is Important

Many organizations rely on test plans that overemphasize functionality and underemphasize validating the system's reliability and resilience. The distribution and complexity of systems makes understanding them more difficult. CE shifts the focus of testing a system from the "happy path" toward testing it under "chaotic path" conditions by intentionally simulating failures. Proactive CE identifies potential system improvements for confidentiality, integrity and availability.

Business Impact

CE is aimed at minimizing time to recovery and the change failure rate, while maximizing uptime and responsiveness. Addressing these elements helps improve customer experience, satisfaction, retention and acquisition. Improving systems reliability also helps traditional cybersecurity concerns of confidentiality, integrity and availability.

Drivers

- Increased complexity of systems and increasing customer expectations are the two largest drivers of CE and the associated tools.
- As systems become richer in features, they also become more complex in their composition and more critical to digital business success.
- Overall, CE helps organizations become more resilient across their processes, knowledge and technology.
- Teams often lack the confidence to handle failures and the psychological safety to take action to resolve incidents. CE can help build that confidence.
- More resilient systems allow support and development teams a better work-life balance, less unplanned work and more consistency in their ability to deliver on planned work.

Obstacles

- Within many organizations, the predominant view of CE is that the practice is random, first implemented during production and increases, rather than reduces, risk.

- Organizational culture and attitudes toward quality and testing can present barriers to adopting CE. When quality and testing are only viewed as overhead costs, there will be a focus on feature development over application reliability.
- It can be challenging just to secure the time and budget to invest in learning CE and associated technologies. Organizations must reach minimum levels of expertise so that value is returned.
- There are costs associated with CE and system reliability that can't be ignored. Not every process in the system demands the same level of resiliency; the focus should be on processes that are most integral to the needs of the business.

User Recommendations

- Utilize a test-environment-first approach by practicing CE in preproduction environments.
- Incorporate CE into your system development, continuous integration/continuous delivery or testing processes.
- Leverage CE when embedding generative AI API calls in your applications to test fallback patterns.
- Implement CE to prepare your organization against ransomware style attacks.
- Utilize scenario-based tests — known as “game days” — to evaluate and learn about how individual IT systems would respond to certain types of outages, including catastrophic failures such as CrowdStrike.
- Prioritize CE activities on critical systems that have elevated security privileges, business-critical services such as payment/payroll or components that are single points of failure.
- Investigate opportunities to use CE in production to facilitate learning and improvement at scale as the practice matures.
- Adopt a platform or tool to track activities and create metrics to build feedback for continuous improvements.

Sample Vendors

Amazon Web Services; Gremlin; Harness; Microsoft; Quinnox; Steadybit

Gartner Recommended Reading

Quick Answer: What Metrics Should We Use to Assess and Improve Software Quality?

Predicts 2023: Observing and Optimizing the Adaptive Organization

Top Strategic Technology Trends for 2023: Digital Immune System

Predicts 2023: How Innovation Will Transform the Software Engineering Life Cycle

Toolkit: Cybersecurity Incident Response Plan

Communities of Practice

Analysis By: Thomas Murphy, Peter Hyde

Benefit Rating: High

Market Penetration: 20% to 50% of target audience

Maturity: Early mainstream

Definition:

A community of practice (CoP) is a stable, informal and voluntary structure designed to foster knowledge sharing, problem solving and organizational learning. These communities promote extensive collaboration and social learning, which are crucial to enable the cultural transformation necessary to drive organizational change.

Why This Is Important

CoPs are crucial for IT organizations, promoting collaboration and breaking down silos to embrace innovative practices. Unlike top-down, policy-focused centers of excellence (COEs), CoPs thrive on motivation, providing a vision and space for collective learning and growth. Through engaging members in meaningful participation, CoPs enhance productivity and adaptability amid market changes and technological advancements.

Business Impact

CoPs significantly enhance business impact by:

- Encouraging knowledge management, professional development and capability improvement

- Shortening the learning curve for employees and improving onboarding
- Providing higher levels of employee satisfaction, leading to better retention, motivation and innovation
- Responding rapidly to customer needs and inquiries
- Reducing duplication of effort
- Spawning new ideas for products and services
- Helping members develop capabilities aligned with organizational needs.

Drivers

- CoPs foster skill development, which is key as organizations need to expand skills in AI, MLOps, ModelOps and prompt engineering.
- The transition from waterfall functional silos to product based organization structures leads to a need for technical and functionally oriented CoPs.
- They support knowledge sharing and learning in hybrid work environments by fostering collaboration in shared areas of interest.
- CoPs can capture and share insights digitally, enhancing continuous learning and onboarding.
- Communities can prevent silos by promoting transversal knowledge management.

Obstacles

- Hybrid work challenges community building, necessitating better leadership skills.
- Sustained efforts also require strong facilitation and agenda skills.
- Hierarchies may resist CoPs, preferring COEs and control.
- CoPs need support from agile, DevOps and innersource practices.
- Management backing and coaching are crucial for CoP success.
- Time pressures, distractions and an emphasis on immediate ROI deprioritize creating space for learning.

User Recommendations

Software engineering leaders should:

- Empower cross-functional communities by providing a clear vision and the necessary time and space for experimentation, collaboration and knowledge sharing.
- Foster a culture of experimentation and risk-taking within CoPs, celebrating failure and new approaches as valuable learning opportunities.
- Enhance CoPs by organizing job rotations, hackathons, and technology conferences to broaden skills and encourage innovation.

Gartner Recommended Reading

Community of Practice Essentials

Case Study: Kick-Starting a Low-Code/No-Code Community of Practice (Heathrow Airport)

Ignition Guide to Building Architecture Communities of Practice

Ignition Guide to Creating Communities of Practice in Software Engineering

Team Collaboration Attributes That Drive High Performance

Event Intelligence Solutions

Analysis By: Matt Crossley, Matthew Brisse

Benefit Rating: Moderate

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

Gartner defines event intelligence solutions (EIS) as the application of AI/machine learning (ML) and data analytics at the event management level to augment, accelerate and automate manual efforts in the event management process. EIS are defined by the key characteristics of cross-domain event ingestion, topology assembly, event correlation and reduction, pattern recognition, and remediation augmentation.

Why This Is Important

The combination of increasing application complexity, monitoring tool proliferation and increasing volumes of events from these tools has shifted the challenge from gathering data to interpreting and responding to it. EIS apply AI/ML and data analytics techniques to classify and cluster cross-domain events in near real time, at scale, and in ways that exceed human capacity. The resulting insights can augment human analysis, accelerate human response or automate a process to help resolve issues.

Business Impact

EIS deliver value through:

- **Agility and productivity:** By reducing alert fatigue through identification and correlation of related events so operators can focus on fewer, yet more relevant and critical events
- **Service availability and triage cost:** By reducing the time and effort required to identify root causes and augmenting, accelerating or automating remediation
- **Increased value from monitoring tools:** By unifying events from siloed tools and learning actionable event patterns across domains

Drivers

Demand for EIS capabilities is accelerating and fueled by:

- **Increasing complexity:** Organizations rely on a growing portfolio of monitoring and observability solutions to ensure the reliability and resilience of increasingly complex and distributed, hybrid and multicloud workloads.
- **Increasing monitoring expectations:** Investments and improvements in monitoring, and the pursuit of observability, generate more data from more sources. Increasing demand and advances in monitoring trends in areas, such as observability and digital experience monitoring, present operators with extremely detailed views into their applications, business services and the end-user experience. Effective use of this additional data requires near-real-time analysis and correlation of events from related assets and services.
- **Demand for reliability:** Shifts in roles and responsibilities are driven by modern operating models, like DevOps and site reliability engineering, in the pursuit of greater availability

and faster incident resolution. EIS enable agility by offloading some mechanical tasks of event triage, root cause analysis and solution identification, accelerating response to common issues and freeing up human creative capacity for novel events and business priorities.

Obstacles

- **Unrealistic expectations:** Hype is a major obstacle to EIS adoption. Clients struggle to separate claims of AI and automation from achievable use cases. This impacts demonstrating the value of EIS, specifically, quantifiable return on investment.
- **Maturity of dependencies:** Benefits of EIS beyond event correlation, such as root cause analysis and remediation, require maturity in dependencies such as change management and automation.
- **Market complexity:** Monitoring and observability vendors are moving up the stack, EIS vendors are reaching into monitoring and observability domains, and AI applications in IT service management allow vendors to use AI capabilities to extend their reach.
- **Disruptive AI:** AI developments continue at pace, resulting in a proliferation of generative AI-based capabilities, moving toward agentic AI solutions and associated expectations. As these developments evolve, expect further changes in the definition of “state of the art,” and equal pressure on existing ways of working.

User Recommendations

- Establish clear, realistic use cases for an EIS pilot, and validate them individually, rather than all at once. This approach helps reveal pockets of potential value that might be missed when evaluating only the aggregate impact. Ultimately, this fundamental step underpins an eventual strategy, while scoping the vendor landscape, clarifying technical and process dependencies, and separating hype from reality.
- Layer the cross-domain analysis of an EIS with a mature monitoring and observability strategy. This approach creates a solid foundation of valuable data for ingestion and analysis, and the surfacing of insights across domains.
- Do not focus solely on automated remediation; this is rarely achieved at scale. Accelerating response and augmenting human decision making has tremendous value.

These approaches often avoid the challenge of the probabilistic uncertainty, combined with automated change in production environments.

Sample Vendors

BigPanda; BMC Software; Dell Technologies; Digitate; Grokstream; Hewlett Packard Enterprise; IBM; Interlink Software; PagerDuty; Selector

Gartner Recommended Reading

[Market Guide for Event Intelligence Solutions](#)

[Infographic: Using AIOps to Manage Operational Telemetry](#)

Climbing the Slope

Application Release Orchestration

Analysis By: Daniel Betts, Hassan Ennaciri

Benefit Rating: High

Market Penetration: More than 50% of target audience

Maturity: Mature mainstream

Definition:

Application release orchestration (ARO) combines deployment automation, pipeline and environment management with release orchestration capabilities to simultaneously improve the quality, velocity and governance of application releases. ARO enables organizations to automate and scale release activities across multiple diverse teams (e.g., DevOps), technologies, development methodologies (e.g., agile), delivery patterns (e.g., continuous), pipelines, processes and toolchains.

Why This Is Important

Demand continues to grow for rapid delivery of products, applications and features to support business agility across multiple technologies. Release is the last stronghold before production and the need to mitigate risk, ensure quality and maintain reliability, while providing release insights across product portfolios has created multiple buyers for ARO

capabilities. These buyers often desperately need ARO's cohesive value, yet are challenged to articulate and/or gain consensus around the business criticality of release activities to drive adoption.

Business Impact

ARO tools provide increased transparency in the release management process by making bottlenecks and wait states visible in areas such as quality and change gates in application development pipelines, infrastructure provisioning and configuration management. Once these constraints are visible and quantifiable, business value decisions can be made to address them and measure improvement. This speeds the realization of direct business value, as new applications, enhancements and bug fixes can be more quickly and reliably delivered.

Drivers

ARO adoption is growing due to demand for:

- **DevOps pervasiveness across the organization:** DevOps adoption into a wide breadth of technologies across the organization including, on-premises, hybrid, cloud, mainframe, packaged applications and edge.
- **Agility and productivity gains:** Faster delivery of new applications and updates in response to changing market demands.
- **Change and release controls:** Demands for change and release management across multiple DevOps workflows.
- **Cost reduction:** Significant reduction of manual interactions by high-skill and high-cost staff, freeing them to work on higher-value activities.
- **Risk mitigation:** From consistent use of standardized, documented processes and configurations across multiple technology domains.
- **Dashboard views:** Outline metrics and predict release quality and throughout, improving time to remediation.
- **Improved visibility and traceability:** Incorporate these into the release process across the organization.

- **Reliability in products and applications:** Adaptive automated release management to protect reliability metrics and measures.

Obstacles

- Adopting release orchestration requires delivery automation maturity that clients are challenged to reach.
- Understanding the migration to automated orchestrated releases from traditional release practices.
- Recognizing that vendors in the DevOps toolchain market build ARO features into platform solutions, with ARO as a subset of capabilities. This can make adoption difficult or hide ARO features' value and the need for a dedicated ARO tool.
- Capturing the release organization space includes challenges, due to feature parity across supplier platforms in the market.

User Recommendations

- Organize DevOps activities into three categories: deployment automation, pipeline and environment management, and release orchestration.
- Explore DevOps platforms that provide ARO, along with other native capabilities, against dedicated ARO tooling solutions.
- Prioritize capabilities for current and future needs before evaluating vendors. When evaluating ARO tools, prioritize tool features and map capabilities to requirements. Legacy environments, where they exist, should be weighted more heavily.
- Simplify and speed up the transition to automated workflows by documenting current application and infrastructure change and release procedures performed by both traditional and DevOps teams.
- Confirm the availability of an ARO platform dashboard with release performance and underlying platform metrics.
- Review integrations by assessing the chosen ARO tooling and its compatibility with your existing toolsets.

Sample Vendors

CloudBees; Digital.ai; Flexagon; GitLab; Microsoft; Planview

Gartner Recommended Reading

[Optimize Your Selection of DevOps Toolchains and Platforms](#)

[Magic Quadrant for DevOps Platforms](#)

[Ignition Guide to Value Stream Mapping of DevOps Process](#)

[Keys to DevOps Success](#)

[Essential Metrics for Agile and DevOps Teams](#)

[Market Guide for DevOps Continuous Compliance Automation Tools](#)

Continuous Quality

Analysis By: Joachim Herschmann

Benefit Rating: High

Market Penetration: 5% to 20% of target audience

Maturity: Adolescent

Definition:

Continuous quality is a systematic approach toward process improvement to achieve the quality goals of business and development. A continuous quality strategy fosters a companywide cultural change to drive quality ownership and engage everyone in achieving the set quality goals. It shifts focus from testing (an activity) to quality and digital immunity (outcomes) and encompasses the practices that help mitigate risks before progressing to subsequent stages of the software development life cycle.

Why This Is Important

The ability to consistently deliver business value with high quality has become critical for organizations seeking to mature their software development processes. Continuous quality encourages a holistic and proactive approach with functional and nonfunctional requirements driving the design, development and delivery of products. Development teams

must use a continuous quality strategy to create quick feedback loops that enable easy adaptation to changes and increase customer satisfaction.

Business Impact

Continuous quality provides a framework for operational excellence that shifts the focus from testing as an activity to a holistic proactive approach to quality. Designing with quality in mind, building in quality and detecting defects earlier allow organizations to release higher-quality products more often than traditional quality control approaches enable. The adoption of a continuous quality strategy can significantly improve an organization's ability to serve and delight its customers.

Drivers

- **User expectations:** Raised end-user expectations for application quality require a shift to a more holistic view of what constitutes superior quality that delights users.
- **Competitive pressure:** Organizations are under the pressure to innovate rapidly in order to launch differentiated products in the market quickly without compromising quality.
- **Fast-moving technology:** Highly dynamic distributed cloud-based architectures require the ability to continuously monitor quality and deal with defects in real time.
- **Holistic quality practices:** As software development scales, teams need consistent delivery practices and a clear understanding of the quality characteristics that make or break a product, especially the nonfunctional (operational) aspects of the product.

Obstacles

- **Lack of clear goals:** Successful continuous quality requires clear goals aligned with the priorities of the business.
- **Internal pushback:** Continuous quality requires engaging stakeholders across the organization and empowering them to be more accountable. Such a holistic approach can be seen as too far-reaching and requires consensus on usage among all team members.
- **Loss of productivity:** Changing organizational culture and engaging in new practices require a significant amount of investment and time. This will impact current timelines and can cause a decrease in productivity prior to reaching steady productivity.

- **Limited to testing only:** Continuous quality includes designing a product with quality in mind, building it with clear quality objectives, and facilitating the discovery of issues early in the development process.

User Recommendations

- Move away from the traditional application- or project-centric focused model to a holistic quality approach by adopting an ecosystem-centric view of quality and placing focus on business outcomes.
- Put a fundamental managed software testing approach in place with test policy and strategy being a focus area.
- Adopt shift-left and shift-right testing practices to extend testing activities across the complete system development life cycle.
- Accelerate product delivery by championing a continuous quality mindset and involving stakeholders across the organization. Allocate ownership and appoint staff with skills needed for continuous quality by identifying the required roles, technologies and practices.
- Enable collaboration with user experience designers and customer experience teams to infuse quality right from the inception of an idea. Establish relevant quality metrics based on the joint objectives that the business and IT are trying to accomplish.

Gartner Recommended Reading

Innovation Insight: Continuous Quality

Infographic: Sketching a Continuous Quality Strategy Over Coffee

Improve Software Quality by Building Digital Immunity

Improve Software Quality and Reduce Security Risks With Shift-Right Testing

Platform Engineering

Analysis By: Bill Blosen, Paul Delory, Cary Pillers

Benefit Rating: Transformational

Market Penetration: 20% to 50% of target audience

Maturity: Early mainstream

Definition:

Platform engineering is the discipline of building and operating a self-service developer platform for software development and delivery. A platform is a layer of tools, services, automations and information maintained as products by a dedicated platform team, designed to support software developers or other engineers by abstracting unnecessary complexity. The goal of platform engineering is to optimize the developer experience and accelerate the delivery of customer value.

Why This Is Important

Digital-empowered enterprises increasingly rely on complex custom software that is required to change rapidly based on customer and internal demands. Software product teams struggle to deliver these changes quickly due to the complexity and variable developer experience. Platform engineering aims to deliver a self-service, curated set of platform capabilities driven by developer priorities to accelerate value delivery in line with internal requirements such as security and architecture.

Business Impact

Platform engineering empowers product teams to deliver software value faster. It reduces the burden of underlying infrastructure construction and maintenance and increases teams' capacity to dedicate time to customer value and learning. Platform engineering voluntarily standardizes the chaos associated with custom software, which results in reduced risk in security, architecture and compliance. It also improves the developer experience, thus reducing employee frustration and attrition.

Drivers

- **Cognitive load:** Adoption of modern, distributed architectural patterns and software delivery practices means that developing and delivering software involves more tools, subsystems and moving parts than ever before. This places a burden on product teams to build a delivery system in addition to the actual software they are trying to produce. Platform engineering reduces the burden of infrastructure construction and maintenance.

- **Scale:** As more teams embrace modern software development practices and patterns, economies of scale are created, whereby there is enough value to justify creating a platform capability shared by multiple teams. This is mostly of value at larger organizations where savings from platform engineering are clearer.
- **Need for increased speed and agility:** The need for speed and agility of software delivery is leading software organizations to pursue DevOps, which is a tighter collaboration of infrastructure and operations (I&O) and development teams, to drive faster delivery and increased deployment frequency. This enables organizations to respond rapidly to market changes, handle workload failures better and tap into new market opportunities. Platform engineering enables this type of cross-team collaboration.
- **Emerging platform construction tools:** Many organizations have built their own platform as homegrown individual efforts tailored to their unique circumstances, including using cloud-native application provisioning platforms and DevOps automation. Platforms generally are not transferable to other companies or sometimes even to other teams within the same company.
- **Emerging developer portals:** A healthy internal developer portal market is developing to front-end the platforms, and Backstage is a popular open-source solution.
- **Infrastructure modernization:** During digital modernization, some forward-looking software development and I&O teams embrace a new platform engineering role as a way to deliver more value, increasing their relevance to the business.

Obstacles

- **Platform engineering is easily misunderstood:** Traditional models of mandated platforms and DevOps toolchains can easily be relabeled and not achieve the true benefits of platform engineering.
- **Lack of skills:** Platform engineering requires solid skills in software engineering, product management and modern infrastructure, all of which are in high demand.
- **Outdated management/governance models:** Many organizations use traditional request-based provisioning models that introduce delays and complexity.
- **Internal politics:** Intraorganizational fights could derail platform engineering. Product teams may resist giving up control of their customized toolchains. There might also be no

appetite to improve the developer experience.

- **Funding:** Enterprises may refuse to fund platform engineering without a clear ROI. Attribution of the costs to user budgets is also tricky. Measuring the benefits/outcomes is critical.
- **Scaling:** It is hard to scale and meet the constant need for evolving the platform.

User Recommendations

- Begin by setting up a platform team with a product owner to guide platform-building efforts with thinnest viable platforms for the complex infrastructure underneath cloud-native and distributed applications including technologies like containers and Kubernetes.
- Enable shift-left and shift-right security within DevOps pipeline platforms, which will provide a compelling paved road to engineers.
- Embed architectural guardrails, compliance controls and any other nonfunctional requirements into the platform to further pave the road for developers.
- Refrain from expecting to buy or build a complete platform, as it is unlikely that any commercially available tool will provide the entirety of the platform you need.
- Implement an internal developer portal, which enables self-service discovery and access to internal developer platform capabilities. Consider Backstage open-source if resources permit or other commercial tools.

Gartner Recommended Reading

[How to Start and Scale Your Platform Engineering Team](#)

[Adopt Platform Engineering to Improve the Developer Experience](#)

[Market Guide for Internal Developer Portals](#)

Product-Centric Delivery Model

Analysis By: Nabeeha Ahmed, Peter Clegg

Benefit Rating: High

Market Penetration: 20% to 50% of target audience

Maturity: Early mainstream

Definition:

The product-centric delivery model features dedicated, multidisciplinary teams focused on continuous customer value through agile practices. The model is defined by its funding model, customer experience focus, emphasis on cross-functional collaboration and distributed decision rights.

Why This Is Important

The ability to quickly adapt to changing market conditions and customer demands is driving organizations to adopt the product-centric delivery model. This model allows organizations to better align with and adapt to changing enterprise priorities. It encourages a shift from the traditional functional organizational structure to embracing multiskilled, persistent teams that collaborate on a product or product line.

Business Impact

A product-centric delivery model enables an enterprise to:

- Focus on outcomes rather than functional outputs, and drive incremental improvements to measurable business outcomes.
- Use venture capital or investment funding models as financial methods to control investment at operational levels.
- Improve agility in response to changing market demands and customer value prioritization.
- Reduce silos, improve collaboration across product or customer value streams and have a flatter organization with more rapid decision making.
- Foster a culture of creativity by encouraging experimentation and innovation, allowing teams to explore new ideas and rapidly iterate on product features.
- Optimize resource allocation and make operations more efficient by enabling teams to focus on specific outcomes and develop deep expertise.

Drivers

- Organizations need to adjust their delivery models to keep pace with market demands and increased volatility.
- Investment and financial models must offer flexibility and facilitate evidence-based market research, aligning with and supporting corporate strategy.
- Organizations are increasingly focusing on delivering value to customers and need rapid, incremental feedback that engineering teams can respond to quickly in order to satisfy and delight customers.
- As organizations grow and add new products or expand existing ones, they need an approach that scales effectively and allows those changes without disrupting the organizational structure.
- Organizations need to establish clear ownership and accountability for product outcomes to ensure high-quality products and motivated teams.
- To improve their product delivery processes, organizations need seamless collaboration across various functions.
- Organizations require an environment that fosters innovation, encouraging exploration of new ideas and technologies to deliver cutting-edge products.

Obstacles

- Inertia from existing organizational culture and management frameworks reluctant to disband current budgets and authority positions.
- Difficulty in finding product management expertise to help overcome change resistance and build effective product structures.
- Lack of alignment between business and IT around outcomes, responsibilities, budgets and success metrics.
- Lack of cohesive leadership support, which results in adoption only in pockets across the organization.
- Outdated governance processes that incentivize control and risk aversion rather than experimentation and innovation.

User Recommendations

- Establish clear goals and objectives for the transition and build leadership support for the necessary culture and governance change.
- Establish a strong partnership between engineering and the business teams as you identify and train product managers, product owners, business leaders and team members on agile and product management practices.
- Adapt governance to embrace business architecture practices such as value stream mapping, business capability modeling, and customer and employee journey mapping.
- Move to a product funding model that allows more rapid and flexible prioritization in response to business demands and changing market conditions.
- Track the delivery of value-based outcomes through product initiatives.
- Manage recurring reviews of outcomes to assess the value of work underway.

Gartner Recommended Reading

[How to Staff Product-Centric Software Delivery Teams: 3 Companies That Got It Right](#)

[Product-Centric Delivery Organizational Structure Library](#)

Immutable Infrastructure

Analysis By: Neil MacDonald, Tony Harvey

Benefit Rating: Moderate

Market Penetration: 5% to 20% of target audience

Maturity: Early mainstream

Definition:

Immutable infrastructure is a process pattern (not a technology) in which the system and application infrastructure, once deployed, are never updated in place. Instead, when changes are required, the infrastructure and applications are updated and redeployed through the continuous integration/continuous delivery (CI/CD) pipeline.

Why This Is Important

Immutable infrastructure ensures the system and application environment, once deployed, remains in a predictable as-deployed state. It simplifies change management, supports faster and safer upgrades, reduces operational errors, improves security, and simplifies troubleshooting. It also enables rapid replication of environments for disaster recovery, geographic redundancy or testing. This approach is easier to adopt with cloud-native applications.

Business Impact

Taking an immutable approach to workload and application management simplifies automated delivery and problem resolution by reducing the options for actions to, essentially, just one — repair or replace the configuration, application or image in the development pipeline and rerelease. The result is an improved security posture and a reduced attack surface with fewer vulnerabilities and a faster time to remediate when new issues are identified.

Drivers

- Linux containers, container registries and Kubernetes are widely adopted. Containers improve the practicality of implementing immutable infrastructure due to their lightweight nature and typically microservices-based architecture, which supports rapid deployment and modular replacement.
- The GitOps deployment pattern, which emphasizes continuously synchronizing the running state to the software repository, has become an effective way to implement immutable infrastructure in Kubernetes-based, containerized environments.
- Infrastructure as code (IaC) tools, including first-party cloud provider IaC tools, have increasingly integrated configuration drift detection and correction, improving the practicality of implementing immutable infrastructure across an application's entire stack and environment.
- Interest has increased in zero-trust and other advanced security postures, where immutable infrastructure can be used to preemptively and proactively regenerate workloads in production from a known good state (assuming compromise), a concept referred to as "systematic workload reprovisioning."
- For cloud-native application development projects, immutable infrastructure simplifies change management, supports faster and safer upgrades, reduces operational errors,

improves security, and simplifies troubleshooting.

Obstacles

- The use of immutable infrastructure requires a strict operational discipline that many organizations haven't yet achieved or have achieved for only a subset of applications.
- IT administrators are reluctant to give up the ability to directly modify or patch runtime systems.
- Applying the immutable infrastructure pattern is most easily done for stateless components. Stateful components, especially data stores, represent special cases that must be handled with care.
- Implementing immutable infrastructure requires a mature automation framework, up-to-date blueprints and bills of materials, and confidence in your ability to arbitrarily recreate components without negative effects on user experience or loss of state.
- Many enterprise applications are stateful applications deployed on virtual machines. These applications are oftentimes commercial off-the-shelf and are not designed for fully automated installation when redeployed.

User Recommendations

- Reduce or eliminate configuration drift by establishing a policy that no software, including the OS, is patched in production. Updates must be made to individual components, versioned in a repository or container registry, and then redeployed.
- Prevent unauthorized change by turning off all administrative access to production compute resources. Examples include not permitting Secure Shell or Remote Desktop Protocol access.
- Adopt immutable infrastructure principles with cloud-native applications first. Cloud-native workloads are more suitable than traditional on-premises or lift-and-shift workloads.
- Treat scripts, recipes and other codes used for infrastructure automation similar to the application source code. This mandates good software engineering discipline, including version control.

- Include immutable infrastructure scripts, recipes, codes and images in your backup and ransomware recovery plans. They will be your primary source to rebuild your infrastructure after an infection.

Sample Vendors

Amazon Web Services; Google; IBM (HashiCorp); JFrog; Microsoft; Perforce; Progress Software; Red Hat; Snyk; Turbot

Gartner Recommended Reading

[Market Guide for Infrastructure Automation and Orchestration Tools](#)

[Market Guide for Cloud-Native Application Protection Platforms](#)

[Reference Architecture Brief: Infrastructure Automation and Orchestration](#)

[DevSecOps Maturity Model for Secure Software Development](#)

[UX Strategy Maturity Model for Software Engineering](#)

Feature Management

Analysis By: Keith Mann

Benefit Rating: High

Market Penetration: 20% to 50% of target audience

Maturity: Early mainstream

Definition:

A feature is a discrete unit of software that provides a single, valuable function or capability within a larger system. Feature management combines the selective enabling or disabling of features via feature flags (also known as feature toggles) with the monitoring, assessment, and comparison of features and feature variants.

Why This Is Important

Feature management enables developers to avoid the technical debt associated with using feature flags at a very large scale, which enables feature flags to be used for more purposes.

Although feature flags were originally used to disable features that were not ready for deployment, they are now also used for experimentation, testing and progressive release. Features are being monitored and tested for value throughout their life cycle, creating feedback loops that aid in feature design.

Business Impact

Almost all businesses can benefit from faster, more reliable delivery of software — the impact of feature management in that sense could be universal. Experimentation has already affected digital marketing and digital consumer product development, where human responses are hard to predict. The biggest potential impact comes from the improved ability to select valuable features for development, based on feedback loops from feature monitoring in production to development planning.

Drivers

- Software engineering teams are mature in their use of agile and DevOps practices. This enables them to deliver software even faster than it can be deployed and consumed. Software engineering organizations can use feature management to decouple delivery from deployment and avoid slowing down teams.
- The demand for continuously available systems is growing due to the rise in online services, mobile computing and critical business applications. This growing demand is driving the need for feature flags and feature management to rapidly recover from defective feature releases.
- With new technologies and corresponding new user experience (UX) options comes uncertainty about which types of experience are most effective for users. Feature management enables experiments that reduce this uncertainty by presenting various experiences to different sample audiences and measuring their responses.
- More organizations are realizing that fast delivery alone does not satisfy stakeholders. They must establish feedback loops from production to feature design to learn which designs produce the greatest value. Feature management enables the value of features to be monitored, tracked and compared.
- Speed-to-market pressures can be met in part through progressive releases. This enables features to be rolled out as their capabilities, such as localization, expand to meet the needs of new user segments.

- The OpenFeature project provides a vendor-agnostic, community-driven standard for feature flagging. This lowers the risk of vendor lock-in and makes it easier for new customers to enter the market.
- As vendors develop their DevOps platform offerings, they are including feature management capabilities, which raises customer awareness and adoption.

Obstacles

- While customers are seeing the importance of feature management and experimentation, their adoption of the capabilities remains relatively low. Vendors must now execute on the vision they have communicated by effective sales and customer success assurance.
- Feature management vendors are shifting from a technical to a business focus in their messaging; however, technology buyers, such as software engineering leaders, often continue to be responsible for justifying the investment.
- As feature management vendors form partnerships with or get acquired by DevOps platform vendors, customers who have chosen different DevOps solutions are uncertain about how feature management vendors will support them.
- Although value feedback loops could be the most important application of feature management, the concept and corresponding practices are still emerging.
- As with many areas of software engineering, low talent availability is a problem. Data scientists and other professionals skilled in experimentation are in short supply and high demand.

User Recommendations

- Ensure that development teams are familiar with feature management techniques. Developers should already be using feature toggles to improve deployment speed and reliability.
- Make your choice of a feature management solution a part of your overall DevOps platform strategy.
- Choose a feature management solution that supports the OpenFeature standard. If your requirements evolve beyond the capabilities of your current solution, this will enable migration to a more capable platform in the future.

- Promote the use of feature management for experimentation and data-driven decision making. This may mean selecting a feature management vendor and training or hiring staff.
- Introduce the concept of value feedback loops to software designers. As tools and practices mature, establish value feedback loops.
- Have software designers clearly identify features during solution design, and define the value of each feature.
- Adopt progressive release practices to reduce change impacts and support earlier delivery.

Sample Vendors

Amplitude; CloudBees; Flagsmith; GitLab; Harness; Kameleoon; LaunchDarkly; Optimizely; Statsig; Unleash

Gartner Recommended Reading

[Software Engineering Teams Must Learn to Deliver More Value](#)

[Adopt Platform Engineering to Improve the Developer Experience](#)

[Improve Continuous Delivery With Feature Management](#)

Appendixes

See the previous Hype Cycle: [Hype Cycle for Agile and DevOps, 2024](#)

Hype Cycle Phases, Benefit Ratings and Maturity Levels

Hype Cycle Phases

Phase	Definition
<i>Innovation Trigger</i>	A breakthrough, public demonstration, product launch or other event generates significant media and industry interest.

<i>Peak of Inflated Expectations</i>	During this phase of overenthusiasm and unrealistic projections, a flurry of well-publicized activity by technology leaders results in some successes, but more failures, as the innovation is pushed to its limits. The only enterprises making money are conference organizers and content publishers.
<i>Trough of Disillusionment</i>	Because the innovation does not live up to its overinflated expectations, it rapidly becomes unfashionable. Media interest wanes, except for a few cautionary tales.
<i>Slope of Enlightenment</i>	Focused experimentation and solid hard work by an increasingly diverse range of organizations lead to a true understanding of the innovation's applicability, risks and benefits. Commercial off-the-shelf methodologies and tools

Source: Gartner (July 2025)

Benefit Ratings

Benefit Rating	Definition
<i>Transformational</i>	Enables new ways of doing business across industries that will result in major shifts in industry dynamics.
<i>High</i>	Enables new ways of performing horizontal or vertical processes that will result in significantly increased revenue or cost savings for an enterprise.
<i>Moderate</i>	Provides incremental improvements to established processes that will result in increased revenue or cost savings for an enterprise.
<i>Low</i>	Slightly improves processes (e.g., improved user experience) that will be difficult to translate into increased revenue or cost savings.

Source: Gartner (July 2025)

Maturity Levels

Maturity Levels	Status	Products/Vendors
<i>Embryonic</i>	In labs	None
<i>Emerging</i>	Commercialization by vendors Pilots and deployments by industry leaders	First generation High price Much customization
<i>Adolescent</i>	Maturing technology capabilities and process understanding Uptake beyond early adopters	Second generation Less customization
<i>Early mainstream</i>	Proven technology Vendors, technology and adoption rapidly evolving	Third generation More out-of-box methodologies
<i>Mature mainstream</i>	Robust technology Not much evolution in vendors or technology	Several dominant vendors

Source: Gartner (July 2025)

independence and objectivity. Its research is produced independently by its research organization without input or influence from any third party. For further information, see "[Guiding Principles on Independence and Objectivity](#)." Gartner research may not be used as input into or for the training or development of generative artificial intelligence, machine learning, algorithms, software, or related technologies.

